

to be optimal, but it is not realizable, because we cannot, in general, accurately predict the behavior of processes. Instead, the strategy serves as a benchmark to which we compare realizable strategies. A well-designed replacement strategy balances the goals of minimal future page faults with the overhead incurred by attempting to predict future page use.

Self Review

1. What other factor complicates replacement strategies on systems that use pure segmentation (as compared with systems that use pure paging)?
2. Is it possible to perform optimal page replacement for certain types of processes? If so, give an example.

Ans: **1)** Such systems must consider the size of the segment being replaced compared to the size of the incoming segment. **2)** Yes, a trivial example is a process with one data page that is intensively referenced and whose data and instructions are referenced purely sequentially.

11.6 Page-Replacement Strategies

In the subsections that follow, we discuss several strategies that determine which page to replace to accommodate an incoming page. Each strategy is characterized by the heuristic it uses to select a page for replacement and the overhead it incurs. Some replacement strategies are intuitively appealing but lead to poor performance due to poor choice of heuristic. Other strategies predict future page usage well, but their overhead can degrade performance. We also discuss how special-purpose hardware can reduce the overhead incurred by page-replacement strategies.

11.6.1 Random Page Replacement

Random (RAND) page replacement is an easy-to-implement, low-overhead page-replacement strategy. Under this strategy, each page in main memory has an equal likelihood of being selected for replacement. One problem with RAND is that it may accidentally select as the next page to replace the page that will be referenced next (which is, of course, the worst page to replace). A benefit of RAND is that it makes replacement decisions quickly and fairly. Since typically there are many page frames from which to choose, there is only a small probability of replacing a page likely to be referenced again almost immediately. Because of its hit-or-miss approach, RAND is rarely used.

Self Review

1. How is RAND fair? Why is this type of fairness inappropriate for replacement strategies?
2. Could RAND ever operate exactly as OPT?

Ans: **1)** RAND is fair in that all pages in memory are equally likely to be replaced. This is inappropriate for replacement strategies, which, to reduce page faults, must try not to replace pages that will be referenced soon. **2)** Yes, it could accidentally make all the right page-

replacement decisions, but it would do this with such miniscule probability that it would be better to answer this question, "No."

11.6.2 First-In-First-Out (FIFO) Page Replacement

In the **first-in-first-out (FIFO) page-replacement** strategy, we replace the page that has been in the system the longest. Figure 11.2 provides a simple example of the FIFO strategy for a process which has been allocated three page frames. The left-most column contains the process's page-reference pattern. Each row in the figure shows the state of the FIFO queue after each new page arrives; pages enter the tail of the queue on the left and exit the head on the right.

Under FIFO page replacement, the system keeps track of the order in which pages enter main memory. When a page must be replaced, the strategy chooses the one that has been in main memory the longest. The intuitive appeal of this strategy seems reasonable—namely, that this page has had its chance and it is time to give another page a chance. Unfortunately, first-in-first-out can replace heavily used pages. For example, on large timesharing systems it is common for many users to share a copy of a text editor as they enter and correct programs. FIFO page replacement on such a system might choose to replace a heavily used editor page. This would be a poor choice, because the page would be recalled to main memory almost immediately, resulting in an increased page-fault rate. Although FIFO can be implemented with relatively low overhead using a queue, it is impractical for most systems. But, as we will see in Section 11.6.7, Modifications to FIFO: Second-Chance and Clock Page Replacement, FIFO forms the basis of various implemented page-replacement schemes.

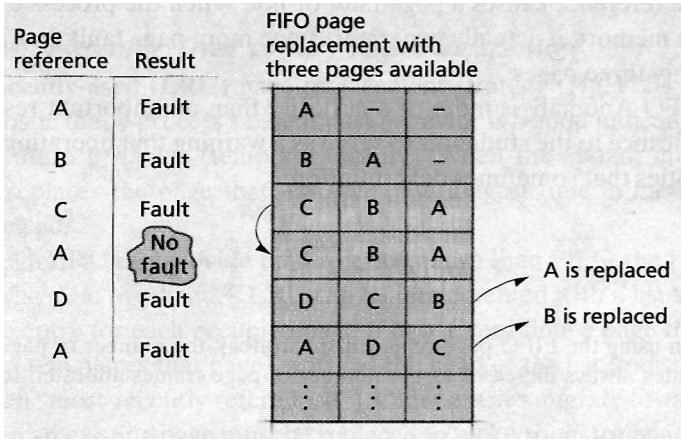


Figure 11.2 | First-in-first-out (FIFO) page replacement.

Self Review

1. Why does FIFO page replacement lead to poor performance for many processes?
2. How does FIFO page replacement compare to OPT for a process executing a loop that references n pages when main memory can hold no more than $n - 1$ of that process's pages?

Ans: 1) FIFO replaces pages according to their age, which, unlike locality, is not a good predictor of how pages will be used in the future. 2) Assume that the pages are numbered from zero to $n - 1$. In this case, when the process references page $n - 1$, FIFO replaces the first page the process referenced in the loop. However, the next page the process references after completing one iteration of the loop is the page that was just replaced. To make room for that page, FIFO replaces the second page the process referenced in the previous iteration of the loop. This is, of course, the next page the process will reference in the current iteration. The optimal strategy would be to replace the page that will be referenced the furthest in the future, which is the page that was just referenced. In this case, OPT would result in one page fault per iteration of the loop, whereas FIFO would result in n page faults per iteration of the loop.

11.6.3 FIFO Anomaly

It would seem reasonable that the more page frames allocated to a process, the fewer page faults the process would experience. Belady, Nelson and Shedler discovered that, under FIFO page replacement, certain page reference patterns actually cause more page faults when the number of page frames allocated to a process is increased.²¹ This phenomenon is called the **FIFO Anomaly** or **Belady's Anomaly**.

Figure 11.3 illustrates an example of the anomaly. The first table demonstrates how the reference pattern causes the system to load and replace pages (using FIFO) when the system allocates three page frames to the process. The second table shows how the system behaves in response to the same reference pattern, but when four page frames have been allocated. To the left of each table, we indicate whether the new page reference causes a page fault or not. When the process executes with four pages in memory, it actually experiences one more page fault than when it executes with only three pages.

The FIFO Anomaly is more of a curiosity than an important result. Perhaps its real significance to the student is to serve as a warning that operating systems are complex entities that sometimes defy intuition.

Self Review

1. (T/F) When using the FIFO page-replacement strategy, the number of page faults a process generates always increases as the number of page frames allocated to that process increases.

Ans: False. The normal behavior is that page faults will decrease because more of the process's pages can be available in memory, decreasing the chance that a referenced page will not be available. Belady's observation is an anomaly; the behavior he observed occurs infrequently.

Page reference	Result	FIFO page replacement with three pages available				FIFO page replacement with four pages available			
A	Fault	A	–	–	Fault	A	–	–	–
B	Fault	B	A	–	Fault	B	A	–	–
C	Fault	C	B	A	Fault	C	B	A	–
D	Fault	D	C	B	Fault	D	C	B	A
A	Fault	A	D	C	No fault	D	C	B	A
B	Fault	B	A	D	No fault	D	C	B	A
E	Fault	E	B	A	Fault	E	D	C	B
A	No fault	E	B	A	Fault	A	E	D	C
B	No fault	E	B	A	Fault	B	A	E	D
C	Fault	C	E	B	Fault	C	B	A	E
D	Fault	D	C	E	Fault	D	C	B	A
E	No fault	D	C	E	Fault	E	D	C	B
Three "no faults"					Two "no faults"				

Figure 11.3 | FIFO anomaly-page faults can increase with page frame allocation.

11.6.4 Least-Recently Used (LRU) Page Replacement

The **least-recently-used (LRU)** page-replacement strategy (Fig. 11.4) relies on the locality heuristic that a process's recent past behavior is a good indicator of the process's near future behavior (temporal locality). When the system must replace a page, LRU replaces the page that has spent the longest time in memory without being referenced.

Although LRU can provide better performance than FIFO, the benefit comes at the cost of system overhead.²² LRU can be implemented with a list structure that contains one entry for each occupied page frame. Every time a page frame is referenced, the system places that page's entry at the head of the list (indicating that the page has been "most-recently referenced"). Older entries migrate toward the tail of the list. When an existing page must be replaced to make room for an incoming one, the system replaces the entry at the tail of the list. The system frees the corresponding page frame (possibly requiring a modified page to be written to secondary stor-

Page reference	Result	LRU page replacement with three pages available		
A	Fault	A	–	–
B	Fault	B	A	–
C	Fault	C	B	A
B	No fault	B	C	A
B	No fault	B	C	A
A	No fault	A	B	C
D	Fault	D	A	B
A	No fault	A	D	B
B	No fault	B	A	D
F	Fault	F	B	A
B	No fault	B	F	A

Figure 11.4 | Least-recently-used (LRU) page-replacement strategy.

age), places the incoming page into that page frame and moves the entry for that frame to the head of the list (because that page is now the one that has been most recently referenced). This scheme would faithfully implement LRU; however, it incurs substantial overhead, because the system must update the list every time a page is referenced.

We must always be careful when applying heuristic reasoning in operating system design; the heuristic—such as LRU here—could fail in certain common situations. For example, the page least recently used could be the next page to be referenced by a program that is iterating inside a loop that references several pages. If the page is replaced, the system will be required to reload it almost immediately.

Self Review

1. (T/F) LRU is designed to benefit processes that exhibit spatial locality.
2. Why is "pure" LRU rarely implemented?

Ans: 1) False. LRU benefits processes that exhibit temporal locality. 2) LRU incurs the overhead of maintaining an ordered list of pages and reordering that list.

11.6.5 ~~Least-Frequently-Used~~ (LFU) Page Replacement

The **least-frequently-used (LFU)** page-replacement strategy makes replacement decisions based on how intensively each page is being used. Under LFU, the system replaces the page that is least frequently used or least intensively referenced. This strategy is based on the intuitively appealing heuristic that a page that is not being intensively referenced is not as likely to be referenced in the future. LFU can be implemented using a counter that is updated each time its corresponding page is referenced, but this can incur substantial overhead.

The LFU page-replacement strategy, too, could easily select incorrect pages for replacement. For example, the least-frequently used page could be the page brought into main memory most recently. This page has been used once, whereas all other pages in main memory may have been used more than once. In this case, the page-replacement mechanism replaces the new page, when in fact the page would be highly likely to be used immediately. Next, we consider a low-overhead page-replacement strategy that makes reasonable decisions most of the time.

Self Review

1. Why is frequency of page usage a poor heuristic for reducing the number of future page faults?
2. Which page-replacement strategy incurs more execution overhead: LRU or LFU?

Ans: 1) Frequency measures the number of times a page is referenced, but it does not indicate how many of those references generated page faults. Because page faults require the process to wait for a page to be loaded from secondary storage, they should be treated with greater weight than references to resident pages. If a reference to a page frequently generates a page fault, then that page is being actively used, and keeping it in memory is likely to reduce future page faults. The same is not always true for resident pages that have been referenced frequently, because frequency does not indicate whether a page is still being actively referenced. 2) The answer depends on the implementation. Both strategies, in their "pure" form, must update page usage on every memory reference, meaning that the two strategies incur similar overhead. LFU updates a counter, while LRU could be rethreading a linked list; the latter would probably involve more overhead.

11.6.6 Not-Used-Recently (NUR) Page Replacement

A popular scheme for approximating LRU with little overhead is the **not-used-recently (NUR)** page-replacement strategy. NUR is based on the idea that a page that has not been used recently is not likely to be used in the near future. The NUR strategy is implemented using the following two hardware bits per page table entry:

- **referenced bit**—set to 0 if the page has not been referenced and set to one if the page has been referenced.
- **modified bit**—set to 0 if the page has not been modified and set to 1 if the page has been modified.

The referenced bit is sometimes called the **accessed bit**. The NUR strategy works as follows. Initially, the system sets the referenced bits of all pages to 0. When

a process references a particular page, the system sets the referenced bit of that page to 1. The modified bits on all pages also are initially set to 0. Whenever a page is modified, the system sets the page's modified bit to 1. When the system must replace a page, NUR first attempts to find a page that has not been referenced (because NUR is intended to approximate LRU). If no such page exists, the system must replace a referenced page. In this case, NUR checks the modified bit to determine whether the page has been modified. If the page has not been modified, the system selects it for replacement. Otherwise, the system must replace a page that has been modified. Recall that replacing a modified page incurs the substantial delay of an additional I/O operation as the modified page is written to secondary storage to preserve its contents. Note, however, that periodic flushing of dirty pages can, on average, reduce or eliminate this delay.

Of course, main memory will likely be actively referenced in a multiuser system, so eventually most or all of the pages' referenced bits will be set to 1. When this is the case, NUR loses the ability to identify the most desirable pages to replace. A technique that has been widely implemented to avoid this problem is for the system periodically to set all the referenced bits to 0, then continue as usual. Unfortunately, this makes even active pages vulnerable to replacement, but only for a brief moment after the bits are reset—active pages will have their referenced bits set to 1 again almost immediately.

Pages can be classified into four groups in the NUR scheme (Fig. 11.5). The pages in the lowest-numbered groups should be replaced first, and those in the highest-numbered groups last. Pages within a group are selected randomly for replacement. Note that Group 2 seems to describe an unrealistic situation—namely, pages that have been modified but not referenced. This occurs because of the periodic resetting of the referenced bits (but not of the modified bits).

Schemes like NUR also can be implemented on machines that lack a hardware referenced bit, and even on machines that lack a hardware modified bit.²³ The referenced and modified bits are typically implemented in hardware and set as part of the execution of each machine instruction. Each of these bits can be simulated by intercepting an operating system's fault handlers and exception handlers as follows.

The referenced bit can be simulated by implementing a corresponding software bit and by initializing each entry in the page table to indicate that the page is not present. When a process references a page and causes a page fault, control reverts to the page fault handler, which sets the referenced bit to 1 and resumes

<i>Group</i>	<i>Referenced</i>	<i>Modified</i>	<i>Description</i>
Group 1	0	0	Best choice to replace
Group 2	0	1	[Seems unrealistic]
Group 3	1	0	
Group 4	1	1	Worst choice to replace

Figure 11.5 | *Page types under NUR.*

normal processing. The modified bit is simulated by marking each page as read-only. When a process attempts to modify the page, a memory-access exception occurs; the exception handler gains control, sets the (software-controlled) modified bit on and changes the access control on that page to read/write. Of course, the mechanism that implements the modified bit must still protect genuine read-only pages from being modified, so the operating system must keep track of which pages are truly read-only and which are truly read/write. The operating system must also periodically set each resident page's referenced bit to zero; the size of this interval is crucial to the performance of NUR. The instructions necessary to perform these actions are most likely to be a small set that can execute quickly. Almost all of today's processors include both a referenced and modified bit to boost the performance of memory managers.

Self Review

1. How does the modified bit improve performance in the NUR replacement strategy?
2. How could NUR replace the worst possible page?
3. How can an NUR page be modified but not referenced?

Ans: **1)** The modified bit enables the operating system to determine which pages can be overwritten without first being flushed to disk. Selecting unmodified pages first reduces I/O when performing page replacement. Note that NUR—to implement its heuristic—nevertheless replaces an unreferenced modified page before it replaces a referenced unmodified page. **2)** The next page that is about to be referenced could have its referenced bit reset to zero just before a page-replacement decision is made. **3)** It truly would be referenced, but NUR periodically resets the referenced bits.

11.6.7 Modifications to FIFO: Second-Chance and Clock Page Replacement

A clear weakness of the FIFO strategy is that it may choose to replace a heavily used page that has been in memory for a long time. This possibility can be avoided by implementing FIFO with a referenced bit for each page and replacing a page only if its referenced bit is set to zero.

The **second-chance** variation of FIFO examines the referenced bit of the oldest page; if this bit is off, the strategy immediately selects that page for replacement. If the referenced bit is on, the algorithm turns off the bit and moves the page to the tail of the FIFO queue. Such a page is treated essentially the same as a new arrival. Over time, the page gradually moves to the head of the queue. When it reaches the head, it will be selected for replacement only if the referenced bit is still off.

Active pages will be selected to return to the tail of the list, because their referenced bits will be set, and will thus remain in main memory. A modified page must be flushed to secondary storage before the system can replace it; so when its referenced bit is set off, the page remains "temporarily unreplaceable" until the system completes the transfer. If a process references this page before the flush is completed, then it is recaptured, thus saving an expensive page-in operation from secondary storage.

The **clock page replacement strategy**, which produces essentially the same results as the second-chance algorithm, arranges the pages in a circular list instead of a linear list.²⁴ Each time a page fault occurs, a list pointer moves around the circular list much as the hand of a clock rotates. When a page's referenced bit is turned off, the pointer is moved to the next element of the list (simulating the movement of this page to the rear of a FIFO queue). The clock algorithm places new arrivals in the first page it encounters with the referenced bit turned off.

Self Review

1. Which strategy incurs more overhead, second chance or clock?
2. Why are second-chance and clock page replacement more efficient than LRU?

Ans: 1) Second chance requires the system to dequeue and requeue a page each time its resident bit is turned off. Clock generally incurs less overhead, because it modifies the value of a pointer each time a page's resident bit is turned off. **2)** These algorithms reduce the number of times the system updates page usage information.

11.6.8 Far Page Replacement

When programs execute, they tend to reference functions and data in predictable patterns. The **far page-replacement strategy** uses graphs to make replacement decisions based on these predictable patterns. The far strategy has been shown mathematically to perform at near-optimal levels, but it is complex to implement and incurs significant execution-time overhead.²⁵⁻²⁶⁻²⁷

The far strategy creates an access graph (Fig. 11.6) that characterizes a process's reference patterns. Each vertex in the access graph represents one of the process's pages. An edge from vertex v to vertex w means that the process can reference page w after it has referenced page v . For example, if an instruction on page v references data on page w , there will be a directed edge from vertex v to vertex w . Similarly, if a function call to page x returns to page y , there will be an edge from vertex x to vertex y . The graph, which can become quite complex, describes how a process can reference pages as it executes. Access graphs can be created by analyzing a compiled program to determine which pages can be accessed by each instruction on each page, which can require significant execution time. [Note: Most studies of the far strategy assume the access graph is constructed before a process is run, although graph construction at runtime has been investigated.]²⁸ The access graph in Fig. 11.6 indicates that, after the process references page B, it will next reference either page A, C, D or E, but it will not reference page G before it has referenced page E.

The replacement algorithm operates in phases much like the clock algorithm. Far initially marks all vertices in the access graph as unreferenced. When the process accesses a page, the algorithm marks as referenced the vertex that corresponds to that page. When the algorithm must select a page for replacement, it chooses the unreferenced page that is furthest away (hence the name "far") from any referenced page in

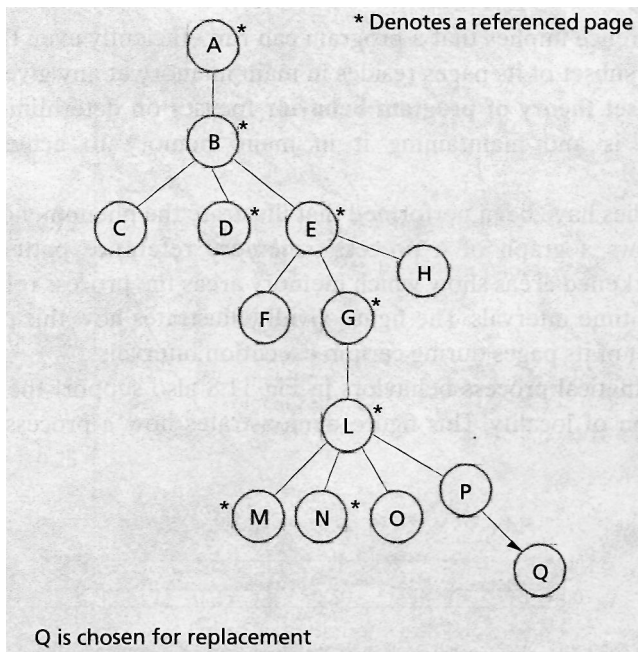


Figure 11.6 | Far page-replacement-strategy access graph.

the access graph (in Fig. 11.6, this corresponds to page Q). The intuitive appeal of this strategy is that the unreferenced page that is furthest from any referenced page is likely to be referenced furthest in the future. If the graph does not contain an unreferenced vertex, the current phase is complete, and the strategy marks all the vertices as unreferenced to begin a new phase.²⁹ At this point, the algorithm replaces the page furthest in the graph from the most-recently referenced page.

The field of graph theory provides algorithms for building and searching the kinds of graphs in the far strategy. However, largely due to its complexity and execution-time overhead, far has not been implemented in real systems.

Self Review

1. Despite providing near-optimal performance, what hinders the far page-replacement strategy from being widely implemented?
2. When might the far strategy replace a page that will be referenced soon?

Ans: 1) Far is complex to implement and it incurs substantial execution-time overhead.
 2) The process may subsequently "walk" the access graph directly to the page that was replaced, at which point it would experience a page fault. This could occur, for example, when a process referenced error-processing routines or issued a series of nested procedure calls.

11.7 Working Set Model

Locality of reference implies that a program can run efficiently even though only a relatively small subset of its pages resides in main memory at any given time. Denning's **working set theory of program behavior** focuses on determining what that favored subset is and maintaining it in main memory to achieve the best performance.^{30,31}

Many studies have been performed that illustrate the phenomenon of locality. Figure 11.7 shows a graph of a process's memory reference pattern across its pages.³² The darkened areas show which memory areas the process referenced during consecutive time intervals. The figure vividly illustrates how this process tends to favor a subset of its pages during certain execution intervals.

The hypothetical process behaviors in Fig. 11.8 also support the existence of the phenomenon of locality. This figure demonstrates how a process's page fault

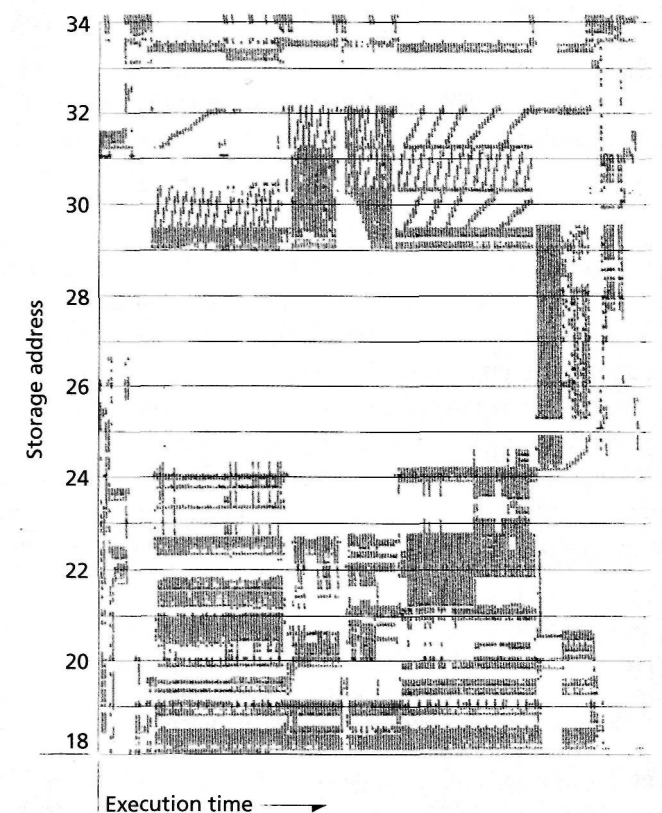


Figure 11.7 | Storage reference pattern exhibiting locality. (Reprinted by permission from IBM Systems Journal. © 1971 by International Business Machines Corporation.)

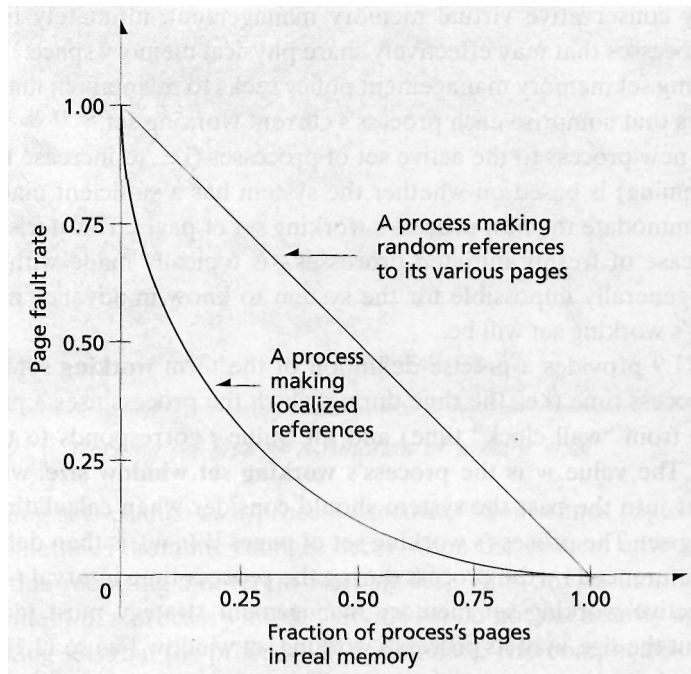


Figure 11.8 | *Dependency of page fault rate on amount of memory for a process's pages.*

rate depends on the amount of main memory available for its pages. The straight line shows how this relationship would appear if the process exhibited a random reference pattern uniformly distributed over all its pages. The curved line shows how processes typically behave. As the number of page frames available to a process decreases, there is an interval over which it does not dramatically affect the page fault rate. But at a certain point, when the number of page frames decreases further, the number of page faults experienced by the running process rises dramatically. The graph shows that the process's page fault rate remains stable as long as its favored subset remains in main memory. However, when the system cannot allocate enough page frames to the process for its favored subset to remain in memory, the process's page fault rate increases dramatically (because the process is constantly referencing the pages that have been replaced).

The principle of locality and the behavior of processes exhibited in the previous figures all support Denning's working set theory of program behavior.^{33,34} This theory asserts that for a program to run efficiently, the system must maintain the program's favored subset (i.e., its working set) of pages in main memory. Otherwise, the system might experience excessive paging activity causing low processor utilization, called **thrashing**, as the process repeatedly requests the same pages from secondary storage.³⁵ One way to avoid thrashing might be to give each process enough

page frames to hold half its virtual space. Unfortunately, rules like this often result in excessively conservative virtual memory management, ultimately limiting the number of processes that may effectively share physical memory space.

A working set memory management policy seeks to maintain in main memory only the pages that comprise each process's current working set.^{36, 37, 38, 39} The decision to add a new process to the active set of processes (i.e., to increase the level of multiprogramming) is based on whether the system has a sufficient main memory space to accommodate the new process's working set of pages. This decision—especially in the case of freshly initiated processes—is typically made with heuristics, because it is generally impossible for the system to know in advance how large a given process's working set will be.

Figure 11.9 provides a precise definition of the term **working set**. The x -axis represents process time (i.e., the time during which the process uses a processor as distinguished from "wall clock" time) and the value t corresponds to the current process time. The value w is the process's **working set window** size, which determines how far into the past the system should consider when calculating the process's working set. The process's working set of pages $W(t, w)$, is then defined as the set of pages referenced by the process during the process-time interval $t - w$ to t .

An effective working set memory management strategy must make careful decisions about the size, w , of its process's working set window. Figure 11.10 illustrates how working set size increases as w increases. This is a consequence of the mathematical definition of working set and is not necessarily an indication of empirically observable working set sizes. The "true" working set of a process is simply the set of pages that must reside in main memory for the process to execute efficiently.

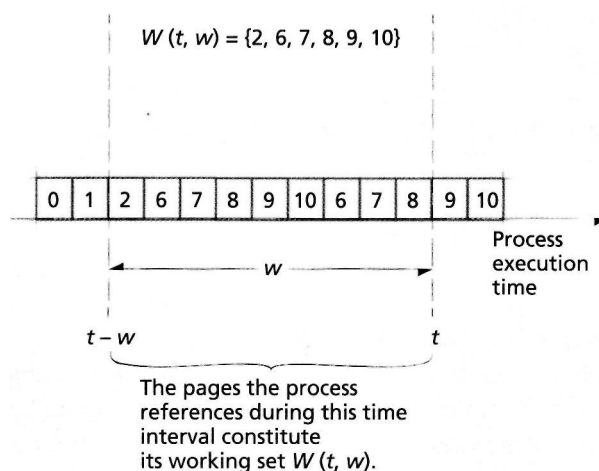


Figure 11.9 | Definition of a process's working set of pages.

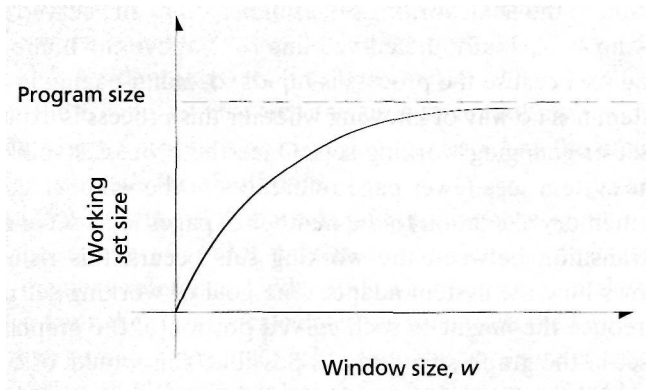


Figure 11.10 | Working set size as a function of window size.

Working sets change as a process executes.⁴⁰ Sometimes pages are added or deleted. Sometimes dramatic changes occur when the process enters a new phase (i.e., execution requiring a different working set). Thus, any assumptions about the size and content of a process's initial working set do not necessarily apply to subsequent working sets that the process will accumulate. This complicates precise memory management under a working set strategy.

Figure 11.11 shows how a process running under a working set memory management strategy might use main memory. First, as the process demand pages in its initial working set one page at a time, the system gradually allocates it enough memory to hold the working set. At this point, the process's memory use stabilizes

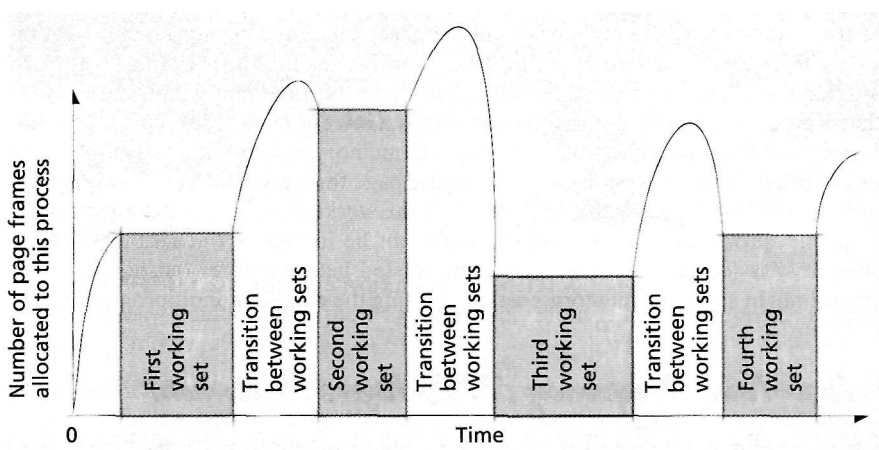


Figure 11.11 | Main memory allocation under working set memory management.

as it actively references the pages in its first working set. The process eventually makes a transition to the next working set, as indicated by the curved line from first to second working set. Initially, the curved line rises above the number of pages in the first working set because the process is rapidly demand-paging in its new working set. The system has no way of knowing whether this process is expanding its current working set or changing working sets. Once the process stabilizes in its next working set, the system sees fewer page references in the window and reduces the process's main memory allocation to the number of pages in its second working set. Each time a transition between the working sets occurs, this rising and falling curved line shows how the system adapts. One goal of working set memory management is to reduce the height of each curved portion of the graph to that of the next working set in the graph as quickly as possible. This would, of course, require that the system rapidly determine which pages from the previous working set (if any) are not part of the process's new working set.

The figure illustrates one of the difficulties with a working set memory management strategy, namely that working sets are transient, and a process's next working set may differ substantially from its current one. A memory management strategy must carefully consider this to avoid overcommitting main memory, which can lead to thrashing. Implementing a true working set memory management policy can incur substantial overhead, especially because the composition of working sets can and does change quickly. Morris discusses the use of custom hardware to make working set storage management more efficient.⁴¹

Self Review

1. Why is it difficult to determine the size of a process's working set?
2. What trade-offs are inherent in choosing a window size, w ?

Ans: 1) If we adhere strictly to Denning's definition, then it is trivial to determine the working set size—it is exactly the number of unique pages that have been referenced within the window, w . If we are concerned with the "true" working set, i.e., the set of pages the process needs to have in memory to run efficiently, then things become more complex. One indication that a process has its working set in memory is a low (or even zero) page fault rate. This could mean that the process has too many pages in memory. If we reduce the number of page frames allocated to the process, at some point the page fault rate will increase, possibly dramatically. Just before it does is the point at which the working set (only) is in memory. 2) If w is too small, a process's true working set might not be in memory at all times, leading to thrashing. If w is too large, memory might be wasted because pages outside of a process' working set might still be in memory, possibly limiting the degree of multiprogramming.

11.8 Page-Fault-Frequency (PFF) Page Replacement

One measure of how well a process is executing in a paging environment is its page-fault rate. A process that faults constantly may be thrashing because it has too few page frames and cannot maintain its working sets in memory. A process that almost never faults may have too many page frames and thus may be impeding the

progress of other processes in the system (or preventing the operating system from increasing the degree of multiprogramming). Ideally, processes should operate at some point between these extremes. The **page-fault-frequency (PFF)** algorithm adjusts a process's **resident page set** (i.e., those pages which are currently in memory), based on the frequency at which the process is faulting.^{42, 43, 44, 45, 46} Alternatively, PFF may adjust a process's resident page set based on the time between page faults, called the process's **interfault time**.

PFF has a lower overhead than working set page replacement because it adjusts the resident page set only after each page fault; a working set mechanism must operate after each memory reference. Under PFF, when a process makes a request that results in a page fault, the strategy calculates the time since the last page fault. If that time is larger than an upper threshold value, then the system releases all pages unreferenced in that interval. If the time is less than a lower threshold value, the incoming page becomes a member of the process's resident page set.

A benefit of PFF is that it adjusts a process's resident page set dynamically, in response to the process's changing behavior. If a process is switching to a larger working set, then it will fault frequently, and PFF will allocate more page frames. Once the process has accumulated its new working set, the page fault rate will stabilize and PFF will either maintain the resident page set or reduce it. A key to the proper and efficient operation of PFF is maintaining the thresholds at appropriate values.

Some systems might increase performance by adjusting their process scheduling algorithms to the frequency with which a process generates page faults. Presumably, processes generating few page faults have accumulated their working sets in main storage. Processes experiencing large numbers of page faults have not yet established their working sets. The conventional wisdom is to favor processes that have established their working sets. Another viewpoint is that processes with high page fault rates should receive priority because they use a processor only briefly before generating an I/O request. A process that is faulting in its next working set appears to be an I/O-bound process. Once the working set is accumulated, the process will "settle down" to "regular" behavior—some processes will be processor bound and others will be I/O bound.

Self Review

1. How does PFF approximate the working set model?
2. What problems could arise if the PFF upper threshold is too small? What if the lower threshold is too large?

Ans: 1) Both PFF and the working set model change the size of a process's allocation space dynamically to prevent thrashing. However, the working set model readjusts after every memory access, whereas PFF readjusts only after each page fault. 2) If the lower threshold were too large, the system would allocate more pages to a process than it needs, which would result in wasted memory. If the upper threshold were too small, the system would release a process's working set pages, leading to thrashing.

11.9 Page Release

Under working set memory management, a process indicates which pages it wants to use by explicitly referencing the pages. Pages that a process no longer requires should be removed from its working set. Under existing memory management strategies, however, needless pages often remain in memory until the management strategy can detect that the process no longer needs them. An alternative strategy would be for the process to issue a **voluntary page release** command to free a page it no longer needs. This would eliminate the delay period caused by letting the process gradually pass the page from its working set.

Voluntary page release could speed program execution for the entire system. One hope in this area is for compilers and operating systems to detect page-release situations, and to do so much sooner than is possible under working set strategies. Realistically, users cannot make such decisions, but applications programmers and systems programmers can.

Self Review

1. Why could voluntary page release yield better performance than a pure working set page-replacement strategy?
2. Why, then, is voluntary page release not widely implemented in today's systems?

Ans: 1) There is a latency in the working set strategy (i.e., the window size, w) that causes pages no longer needed to "hang around" before being replaced. Voluntary page release could release those pages sooner, leading to more efficient use of memory. 2) The real issue is whether it is indeed possible to choose the right pages to release. This is hard to do, because we know that we cannot, in the general case, predict the path of execution a program will take.

11.10 Page Size

An important characteristic of a paged virtual memory system is the size of the pages and page frames that the system supports. In today's systems, there is no one "industry standard" page size, and many architectures support multiple page sizes, with demonstrable improvements in performance.⁴⁷ When choosing a page size (or page sizes), the system designer should evaluate several concerns, based on the goals and limitations of the system to be designed.

Many early results in the literature, both theoretical and empirical, point to the need for small pages.^{48-49,50} As both memory and program sizes increase rapidly, larger page sizes have become more desirable. What considerations determine whether a page should be large or small? Several are summarized here:

- A large page size increases the range of memory that the TLB can reference with each entry. This increases the likelihood of TLB hits, which improves dynamic address translation performance.⁵¹
- In general, a large page size can reduce the number of time-consuming I/O operations that transfer information between main memory and secondary

storage. A system transferring such information using a small page size may require several separate I/O operations, which would increase a process's space-time product (see Section 11.3). However, anticipatory paging and (as we discuss in the next chapter) disk scheduling algorithms can reduce this overhead.

- Processes tend to exhibit locality of reference over a small portion of their address spaces, so a smaller page size would help a process establish a smaller, tighter working set, leaving more memory available to other processes.⁵²
- A small page size leads to a large number of pages and page frames and correspondingly larger page tables. As discussed in Section 10.4.4, Multi-level Page Tables, these tables can consume a significant portion of main memory (i.e., table fragmentation). Large page sizes reduce table fragmentation by decreasing the number of page table entries (at the cost of increased internal fragmentation).
- In a combined segmentation/paging organization the system may experience internal fragmentation, because procedure and data units rarely comprise an integral number of pages, so a segment is just as likely to have its last page nearly full as nearly empty (Fig. 11.12). Thus, each segment contains, on average, one-half page of internal fragmentation. More fragmentation results in a larger average working set size for programs.⁵³ The system can reduce the amount of internal fragmentation by employing smaller page sizes.⁵⁴

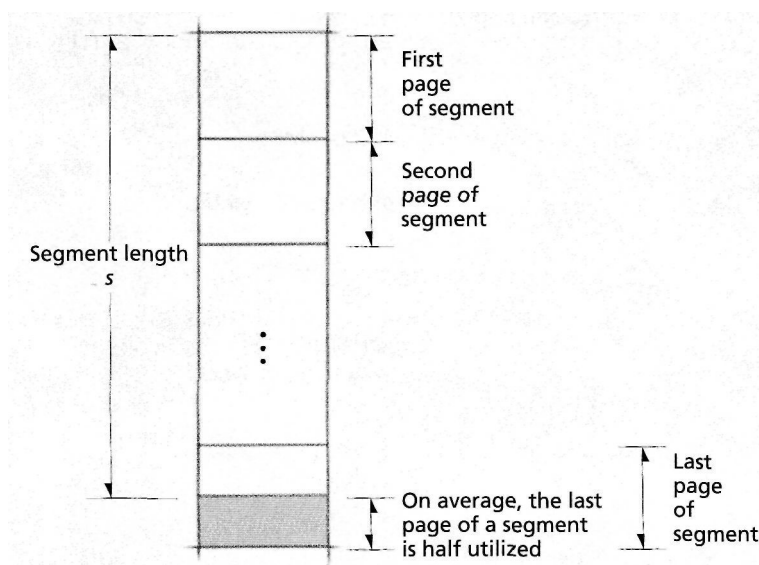


Figure 11.12 | Internal fragmentation in a paged and segmented system.

Although supporting multiple page sizes can address many of the limitations of both large and small page sizes, it has several disadvantages. First, operating systems require hardware support for multiple page sizes to provide efficient dynamic address translation. This means that the page address translation hardware must be able to accommodate large and small pages. In the Intel Pentium 4 processor, this is accomplished using multiple TLBs, each of which is dedicated to storing PTEs for one page size. This added hardware complexity increases cost. Also, multiple page sizes introduces the possibility of external fragmentation similar to that in segmentation systems, because blocks are of variable size.⁵⁵

Figure 11.13 lists the page sizes used by several systems. Note the relatively small single page sizes chosen for earlier computers, whereas more recent computers support larger page sizes and multiple page sizes.

Self Review

1. Why are large page sizes more favorable in today's systems than they were decades ago?
2. What are the negatives of having multiple page sizes?

Ans: 1) The cost of memory has become cheaper, leading to systems that contain large memories and applications that require increasingly large amounts of memory. In today's systems, the cost of internal fragmentation due to larger page sizes is of less concern than it was

<i>Manufacturer</i>	<i>Model</i>	<i>Page Size</i>	<i>Real address size</i>
Honeywell	Multics	1KB	36 bits
IBM	370/168	4KB	32 bits
DEC	PDP-10 and PDP-20	512 bytes	36 bits
DEC	VAX 8800	512 bytes	32 bits
Intel	80386	4KB	32 bits
Intel/AMD	Pentium 4 / Athlon XP	4KB or 4MB	32- or 36 bits
Sun	UltraSparc II	8KB, 64KB, 512KB, 4MB	44 bits
AMD	Opteron / Athlon 64	4KB, 2MB and 4MB	32, 40, or 52 bits
Intel-HP	Itanium, Itanium 2	4KB, 8KB, 16KB, 64KB, 256KB, 1MB, 4MB, 16MB, 64MB, 256MB	Between 32 and 63 bits
IBM	PowerPC 970	4KB, 128KB, 256KB, 512KB, 1MB, 2MB, 4MB, 8MB, 16MB, 32MB, 64MB, 128MB, 256MB	32 or 64 bits

Figure 11.13 | Page sizes in various processor architectures.^{56, 57, 58}

decades ago, when memory was far more expensive. Large pages also require the system to perform fewer costly I/O operations to load a large portion of a process's virtual address space into memory. However, there are many cases in which the operating system must store data which is only a fraction of the page size. In this case, multiple page sizes might improve memory utilization, at the cost of additional memory management overhead. 2) Both the operating system and hardware must support multiple page sizes to provide efficient memory management. Operating systems must be rewritten so that they efficiently manage multiple page sizes, which requires costly software development. Processors must increase in complexity to support multiple page sizes, which tends to increase their cost.

11.11 Program Behavior under Paging

Many studies have been performed examining the behavior of processes in paging environments.^{59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80} In this section we

present some qualitative results of these studies.

Figure 11.14 shows the percentage of a hypothetical process's pages that have been referenced, starting from the time the process begins execution. The initial sharp upward slope indicates that a process tends to reference a significant portion of its pages immediately after execution begins. With time, the slope diminishes, and the graph asymptotically approaches 100 percent. Certainly some processes reference 100 percent of their pages, but the graph is drawn to reflect that many processes may execute for a long time without doing so. This is the case, for example, when certain error-processing routines are rarely invoked.

The number of faults a process experiences depends on the size and behavior of the system's processes. If they have small working sets, the number of page faults experienced by a running process tends to increase as the page size increases. This phenomenon occurs because, as page size increases, more procedures and data that will not be referenced are brought into a fixed-size main memory. Further, as page

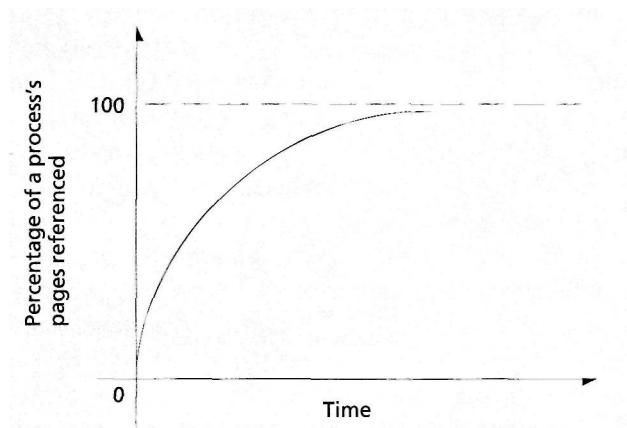


Figure 11.14 | Percentage of a process's pages referenced with time.

size continues to increase, the system incurs more internal fragmentation. Thus, a smaller percentage of a process's limited main memory is occupied by procedures and data that will be referenced. A process with a large working set requires a greater number of small page frames, which can lead to a large number of page faults each time the process runs. If a process's working set contains pages that are contiguous in the process's virtual address space, the number of page faults tends to decrease as the page size increases. This occurs because each page fault loads into memory a significant portion of the process's working set.

Figure 11.15 shows how the average interfault time (i.e., the time between page faults) varies as the number of page frames allocated to a process increases. The graph is nondecreasing—the more page frames a process has, the longer the time between page faults (subject, of course, to occasional strange behavior such as the FIFO anomaly).⁸¹ At one point, the graph bends, and its slope declines sharply. At this point the process has its entire working set in main memory. Initially, the interfault time grows quickly, as more of the working set resides in main memory. Once the physical memory allotment is sufficient to hold the process's working set, the curve bends sharply, indicating that the effect of allocating additional page frames on increasing the interfault time is not as great. Again, the goal of a memory management strategy should be to maintain the working set in main memory.

The qualitative discussions of this section generally point to the validity of the working set concept. As computer architecture and software design evolves, these results will need to be reevaluated.

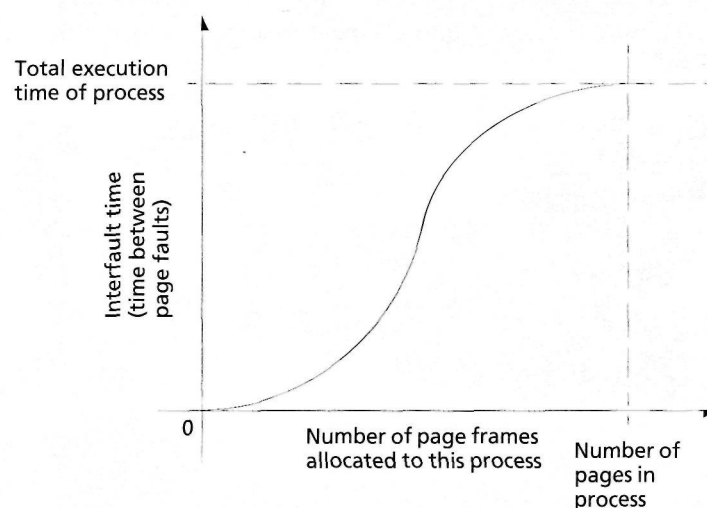


Figure 11.15 | Dependency of interfault time on the number of page frames allocated to a process.

Self Review

1. How does program behavior under paging support the notion of lazy allocation, whereby the operating system does not allocate page frames to a process until the process explicitly references its pages?
2. (T/F) The interfault time for a particular process always increases as the number of page frames allocated to a process increases.

Ans: 1) Figure 11.14 demonstrates that the operating system would waste considerable memory if it reserved memory that a process would not use until later in its execution. However, if a process references its pages with predictable patterns, anticipatory paging could improve performance by reducing page-wait times. 2) False. This is indeed the normal behavior, but if the algorithm is subject to Belady's Anomaly, the interfault time might decrease.

11.12 Global vs. Local Page Replacement

When evaluating the page-replacement strategies presented in the previous sections, we provided reference patterns for a single, arbitrary process. In most real systems, the operating system must provide a page-replacement strategy that yields good performance for a number of diverse processes.

When implementing a paged virtual memory system, the operating system designer must decide whether page-replacement strategies should be applied to all processes as a unit (i.e., global strategies) or consider each process individually (i.e., local strategies). Global page-replacement strategies tend to ignore the characteristics of individual process behavior; local page-replacement strategies enable the system to adjust memory allocation according to the relative importance of each process to improve performance.

As we will see, a heuristic that yields good results when applied to individual process behavior can lead to poor performance when applied to the system as a whole. There is no conventional wisdom on this—Linux, for example, implements a global page-replacement strategy and Windows XP implements a local page-replacement strategy (see Section 20.6.3, Page Replacement, and Section 21.7.3, Page Replacement, respectively).⁸²

The **global LRU (gLRU)** page-replacement strategy replaces the least-recently-used page in the entire system. This simple strategy does not attempt to analyze individual process behavior or the relative importance of processes when selecting a page to replace. Consider, for example, a system that schedules processes using round robin—the next process to run is the process that has waited the longest. In this case, pages belonging to the next process to run are often the least-recently-used pages, so gLRU is likely to replace the next page to be used. Note that gLRU can result in poor performance regardless of individual process behavior if the system implements a variant of the round-robin scheduling algorithm.

Glass and Cao suggest the SEQ (sequence) global replacement strategy, which is a modified version of LRU that adjusts its strategy based on process behavior.⁸³ In general, the SEQ strategy uses the LRU strategy to replace pages. As discussed in Section 11.6.4, LRU performs poorly when processes enter loops that

reference a sequence of pages that cannot all fit in memory at once. In this case, the optimal page to replace is the page that was most-recently used because it will be referenced furthest in the future (i.e., during the next iteration of the loop). Accordingly, the SEQ strategy uses the heuristic that a process experiencing a sequence of page faults when referencing a series of contiguous pages is exhibiting looping behavior. When such behavior is detected, it applies the most-recently used (MRU) strategy, which is optimal for that reference pattern. If a process experiences page fault when referencing a noncontiguous page, SEQ uses the LRU strategy until another sequence of page faults to contiguous pages is detected.

11.13 Case Study: Linux Page Replacement

In this section we discuss the page-replacement strategy implemented in Linux. More information about memory management in Linux can be found in Section 20.6, Memory Management. When physical memory is full and nonresident data is requested by processes or the kernel, page frames must be freed to fill the request. Pages are divided into active pages and inactive pages. To be considered active, a page must have been referenced recently. One goal of the memory manager is to maintain the current working set inside the collection of active pages.⁸⁴

Linux uses a variation of the clock algorithm to approximate an LRU page-replacement strategy (Fig. 11.16). The memory manager uses two linked lists: the active list contains active pages, the inactive list contains inactive pages. The lists are organized such that the most-recently used pages are near the head of the active list, and the least-recently used pages are near the tail of the inactive list.⁸⁵

When a page is first brought into memory, it is placed in the inactive list and is marked as having been referenced by setting its referenced bit. The memory manager periodically determines whether the page has been subsequently referenced, such as during a page fault. If the page is active or inactive and its referenced bit is off, the bit is turned on. Similar to the clock algorithm, this technique ensures that recently referenced pages are not selected for replacement.

Otherwise, if the page is inactive and is being referenced for the second time (its referenced bit is already on), the memory manager moves the page to the head of the active list, then clears its referenced bit.⁸⁶ This allows the kernel to determine whether a page has been referenced more than once recently. If so, the page is placed in the active list so that it is not selected for replacement. To ensure that the active list contains only pages that are being heavily referenced, the memory manager periodically moves any unreferenced pages in the active list to the head of the inactive list.

This algorithm is repeated until the specified number of pages have been moved from the tail of the active list to the head of the inactive list. A page in the inactive list will remain in memory until it is selected for replacement. While a page is in the active list, however, it cannot be selected for replacement.⁸⁷

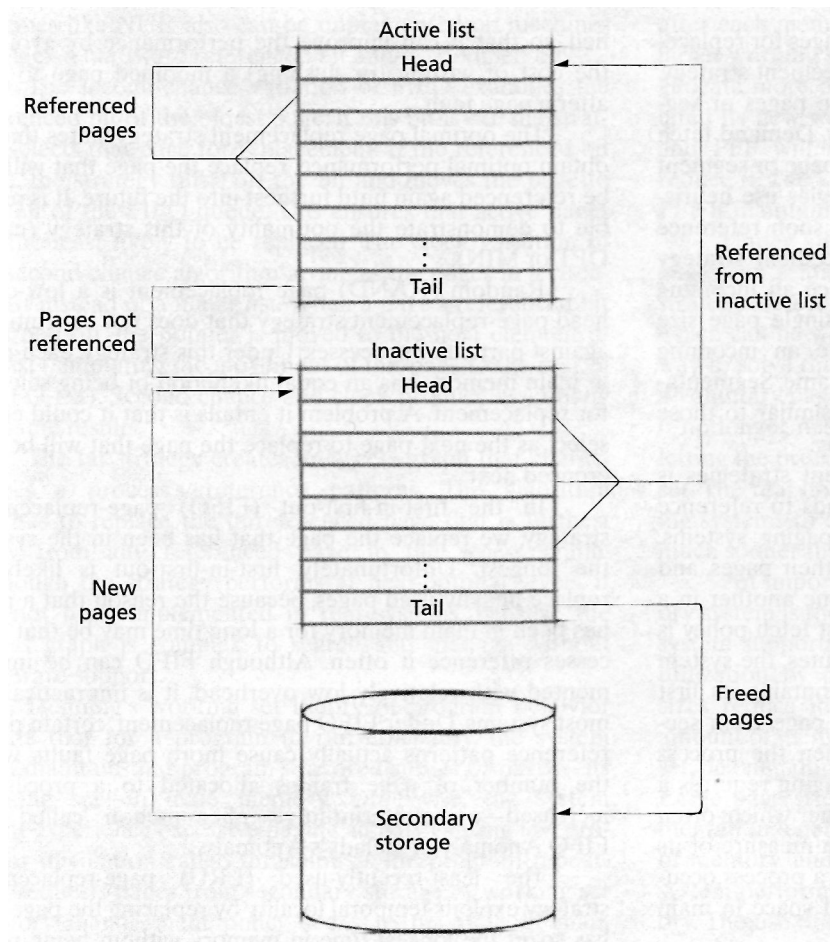


Figure 11.16 | Linux page replacement overview.

Web Resources

publib16.boulder.ibm.com/pseries/en_US/aixbman/admnconc/vtm_overview.htm

Describes virtual memory management in AIX, which is IBM's UNIX-based system.

www.csn.ul.ie/~mel/projects/vm/

Describes virtual memory management for the Linux 2.4 kernel.

www.informit.com/isapi/product_id~%7B6D5ACA10-5AB0-4627-81E5-65B9F6E080AC%7D/content/index.asp

Describes virtual memory management in Windows 2000 running on an Intel x86 processor.

developer.apple.com/documentation/mac/Memory/Memory-2.html

Describes virtual memory management in the Apple Macintosh operating system.

msdn.microsoft.com/library/default.asp?url=/library/en-us/appendix/hh/appendix/enhancements5_3oc3.asp

Discusses memory management enhancements in Windows XP.

www.nondot.org/~sabre/os/articles/MemoryManagement/
Lists links to several articles related to memory management.

Summary

The technique a system employs to select pages for replacement when memory is full is called a replacement strategy. A system's fetch strategy determines when pages or segments should be loaded into main memory. Demand fetch strategies wait for a process to reference a page or segment before loading it. Anticipatory fetch strategies use heuristics to predict which pages a process will soon reference and load those pages or segments. A placement strategy determines where in main memory to place an incoming page or segment. Paging systems with a single page size trivialize the placement decision, because an incoming page may be placed in any available page frame. Segmentation systems require placement strategies similar to those used in variable-partition multiprogramming.

Central to most memory management strategies is the concept of locality—that a process tends to reference memory in highly localized patterns. In paging systems, processes tend to favor certain subsets of their pages, and these pages often tend to be adjacent to one another in a process's virtual address space. The simplest fetch policy is demand paging—when a process first executes, the system loads into main memory the page that contains its first instruction. After that, the system loads a page from secondary storage to main memory only when the process explicitly references that page. Demand paging requires a process to accumulate pages one at a time, which often increases a process's space-time product—a measure of its execution time (i.e., the duration for which a process occupies memory) multiplied by the amount of space in main memory it occupies.

In anticipatory paging, the operating system attempts to predict the pages a process will need and preloads these pages when memory space is available. Anticipatory paging strategies must be carefully designed so that overhead incurred by the strategy does not reduce system performance. A demand prepaging strategy preloads a group of pages into memory when a process references a nonresident page, which can be effective when processes exhibit spatial locality.

When a process generates a page fault, the memory management system must locate the referenced page in secondary storage, load it into a page frame in main memory and update the corresponding page table entry. Most modern architectures support a modified (or dirty) bit in their page table entries; this bit is set to 1 if the page has been modified and 0 otherwise. The operating system uses this bit to quickly determine which pages have been modi-

fied, so that it can improve the performance by avoiding the cost of writing (or flushing) a modified page to disk after a page fault.

The optimal page replacement strategy states that, to obtain optimal performance, replace the page that will not be referenced again until furthest into the future. It is possible to demonstrate the optimality of this strategy (called OPTorMIN).

Random (RAND) page replacement is a low-overhead page-replacement strategy that does not discriminate against particular processes. Under this strategy, each page in main memory has an equal likelihood of being selected for replacement. A problem it entails is that it could easily select as the next page to replace the page that will be referenced next.

In the first-in-first-out (FIFO) page-replacement strategy, we replace the page that has been in the system the longest. Unfortunately, first-in-first-out is likely to replace heavily used pages, because the reason that a page has been in main memory for a long time may be that processes reference it often. Although FIFO can be implemented with relatively low overhead, it is impractical for most systems. Under FIFO page replacement, certain page-reference patterns actually cause more page faults when the number of page frames allocated to a process is increased—a counterintuitive phenomenon called the FIFO Anomaly or Belady's Anomaly.

The least-recently-used (LRU) page-replacement strategy exploits temporal locality by replacing the page that has spent the longest time in memory without being referenced. Although LRU can provide better performance than FIFO, the benefit comes at the cost of increased system overhead. Unfortunately, the page least-recently used could be the next page to be referenced by a program that is iterating inside a loop that references several pages.

The least-frequently-used (LFU) page-replacement strategy replaces the page that is least intensively referenced. This strategy is based on the heuristic that a page not referenced often is not likely to be referenced in the future. LFU could select the wrong page for replacement; for example, the least-frequently-used page could easily be the page brought into main memory most recently—a page that any reasonable page-replacement strategy would like to keep in main memory.

The not-used-recently (NUR) replacement strategy approximates LRU with little overhead by using a referenced bit and a modified bit to determine which page has

not been used recently and can be replaced quickly. Schemes like NUR also can be implemented on machines that lack a hardware referenced bit and/or modified bit.

The second-chance variation of FIFO examines the referenced bit of the oldest page; if this bit is off, the strategy selects that page for replacement. If the referenced bit is on, the strategy turns off the bit and moves the page to the tail of the FIFO queue. This ensures that active pages are the least likely to be replaced. The clock variation of the second-chance algorithm arranges the pages in a circular list instead of a linear list. When a page's referenced bit is turned off, the pointer is moved to the next element of the list (simulating the movement of this page to the rear of a FIFO list). Second chance and clock produce essentially the same results.

The far strategy creates an access graph that characterizes a process's reference patterns. The algorithm chooses to replace the unreferenced page that is furthest away from any referenced page in the access graph. Although the strategy performs at near-optimal levels, it has not been implemented in real systems, because the access graph is complex to search and manage without hardware support.

Denning's working set theory of program behavior asserts that for a program to run efficiently, the system must maintain that program's favored subset of pages—its working set—in main memory. Otherwise, the system might experience excessive paging activity causing low processor distillation called thrashing as the program repeatedly requests pages from secondary storage. A working set memory management policy seeks to maintain in main memory only the pages that comprise each process's current working set. The decision to add a new process to the active set of processes is based in part on whether the system has sufficient main memory space to accommodate the new process's working set of pages. A process's working set window size specifies how far into the past the system should consider when calculating the process's working set. One challenge of working set memory management is that working sets are transient, and a process's next working set may differ substantially from its current one.

The page-fault-frequency (PFF) algorithm adjusts a process's resident page set (i.e., those pages which are currently in memory), based on the frequency at which the process is faulting. Alternatively, PFF may adjust a process's resident page set based on the time between page faults, called the process's interfault time. An advantage of PFF over working set page replacement is lower overhead—PFF adjusts the resident page set only after each

page fault, whereas a working set mechanism must operate after each memory reference. If a process is switching to a larger working set, then it will fault frequently and PFF will allocate more page frames. Once the process has accumulated its new working set, the page fault rate will stabilize and PFF will either maintain the resident page set or reduce it. The key to the proper and efficient operation of PFF is maintaining the thresholds at appropriate values.

Under all memory management strategies, needless pages can remain in main memory until the management strategy detects that the process no longer needs them, which can be well after the page is no longer needed. One way to solve this problem would be for the process to issue a voluntary page release to free a page frame that it knows it no longer needs, eliminating the delay period caused by letting the process gradually pass the page from its working set. The real hope in this area is for compilers and operating systems to detect page-release situations, and to do so much sooner than is possible under working set strategies.

An important characteristic of a paged virtual memory system is the size of the pages and page frames that the system supports. Some systems improve performance and utilization by providing multiple page sizes. Small page sizes reduce internal fragmentation and can reduce the amount of memory required to contain a process's working set, leaving more memory available to other processes. Large page sizes reduce wasted memory from table fragmentation, enable each TLB entry to map a larger region of memory, and reduce the number of I/O operations the system performs to load a process's working set into memory. The use of multiple page sizes requires both hardware and software support, which can be costly to implement, and introduces the possibility of external fragmentation due to variable page sizes.

Processes tend to reference a significant portion of their pages within a short time after execution begins, then access most (or all) of their remaining pages at a slower rate. The average interfault time (i.e., the time between page faults) monotonically increases in general—the more page frames a process has, the longer the time between page faults (subject, of course, to strange behavior such as that observed in the FIFO Anomaly).

When implementing a paged virtual memory system, the operating system designer must decide whether page-replacement strategies should be applied to all processes as a unit (i.e., global strategies) or consider each process individually (i.e., local strategies). Global page-replacement strategies tend to ignore the characteristics of individual process behavior; local page-replacement strategies enable the sys-

tem to adjust memory allocation according to the relative importance of each process to improve performance. The global LRU (gLRU) page-replacement strategy replaces the least-recently-used page in the entire system. The SEQ

(sequence) global replacement strategy, uses the LRU strategy to replace pages until sequence of page faults to contiguous pages is detected, at which point it uses the most-recently-used (MRU) page-replacement strategy.

Key Terms

accessed bit—See referenced bit.

anticipatory paging—Technique that preloads a process's non-resident pages that are likely to be referenced in the near future. Such strategies attempt to reduce the number of page faults a process experiences.

Belady's Anomaly—See FIFO Anomaly.

clock page-replacement strategy—Variation of the second-chance page-replacement strategy that arranges the pages in a circular list instead of a linear list. A list pointer moves around the circular list, much as the hand of a clock rotates, and replaces the page nearest the pointer (in circular order) that has its referenced bit turned off.

demand paging—Technique that loads a process's nonresident pages into memory only when the process explicitly references the page.

dirty bit—Page table entry bit that specifies whether the page has been modified (also known as the modified bit).

far page-replacement strategy—Graph-based page-replacement strategy that analyzes a program's reference patterns to determine which page to replace. This strategy replaces the page that is furthest from any referenced page in the graph and that has not been referenced recently.

FIFO Anomaly—Phenomenon in FIFO page-replacement strategy whereby increasing a process's page frame allocation increases the number of page faults it experiences; normally, page faults should decrease as more page frames become available.

first-in-first-out (FIFO) page replacement—Page-replacement strategy that replaces the page that has been in memory longest. FIFO incurs low overhead but generally does not predict future page usage accurately.

flushing a page—Copying the contents of a modified page in main memory to secondary storage so another page can be placed in its frame. When this occurs, the page's dirty bit is cleared, which enables the operating system to quickly determine that the page can be overwritten by an incoming page, which can reduce page-wait times.

global least-recently-used (gLRU) page replacement—Global page-replacement strategy that replaces the page that has

not been referenced for the longest time in the entire system. LRU can perform poorly because variants of round-robin scheduling cause the system to exhibit a large-scale looping reference pattern. The SEQ variant of gLRU page replacement attempts to improve performance by replacing the most-recently-used page when it detects a looping reference pattern.

interfault time—Time between a process's page faults. This is used in the page-fault-frequency page-replacement strategy to determine when to increase or decrease a program's page frame allocation.

least-frequently-used (LFU) page replacement—Page-replacement strategy that replaces the page that is least frequently used or least intensively referenced. LFU is easy to implement, but generally does not predict future page usage well.

least-recently-used (LRU) page replacement—Page-replacement strategy that replaces the page that has not been referenced for the longest time. LRU generally predicts future page usage well but incurs significant overhead.

modified bit—Page table entry bit that indicates whether a page has been modified and hence must be copied to secondary storage before being replaced (also known as the dirty bit).

not-used-recently (NUR) page replacement—Low-overhead approximation to the LRU page-replacement strategy: uses referenced bits and dirty bits to replace a page. NUR first attempts to replace a page that has not been referenced recently and that has not been modified. If no such page is available, the strategy replaces a dirty page that has not been referenced recently, a clean page that has been referenced recently or a referenced page that has been referenced recently, in that order.

optimal (OPT) page replacement—Unrealizable page-replacement strategy that replaces the page that will not be used until furthest in the future. This strategy has been shown to be optimal.

page-fault-frequency (PFF) page replacement—Algorithm that adjusts a process's resident page set based on the frequency with which the process is faulting. If a process is

switching to a larger working set, then it will fault frequently, and PFF will allocate more page frames. Once the process has accumulated its new working set, the page fault rate will stabilize, and PFF will either maintain the resident page set or reduce it. The key to the proper and efficient operation of PFF is maintaining the thresholds at appropriate values.

page-replacement strategy—Strategy that determines which page to replace to provide space for an incoming page. Page replacement strategies attempt to optimize performance by predicting future page usage.

prefetching—See anticipatory paging.

prepaging—See anticipatory paging.

random (RAND) page replacement—Page-replacement strategy in which each page in main memory has an equal likelihood of being selected for replacement. Although this strategy is fair and incurs little overhead, it does not attempt to predict future page usage.

referenced bit—Page table entry bit indicating whether a page has been referenced recently. Several strategies reset this bit to more accurately determine how recently a page has been referenced.

resident page set—Set of a process's pages that are currently in memory; these pages may be referenced without generating a page fault. The resident page set might differ in size from a process's working set, which is the set of pages that must be in memory for a process to execute efficiently.

second-chance page-replacement strategy—Variation of FIFO page replacement that uses the referenced bit and a FIFO queue to determine which page to replace. If the oldest page's referenced bit is off, second chance replaces the page; otherwise it turns off the referenced bit on the oldest page and moves it to the tail of the FIFO queue. If its referenced bit is on, the strategy turns off the bit and examines the next page or pages until it locates a page with its referenced bit turned off.

space-time product—Value that measures the product of a process's execution time (i.e., the duration for which a process occupies memory) and the amount of real-memory space the process occupies. Ideally, memory management strategies should reduce this quantity to increase a system's degree of multiprogramming.

spatial locality—Empirical property that, in paging systems, states that processes tend to favor certain subsets of their pages, and that these pages tend to be near one another in a process's virtual address space. A process accessing sequential indices of an array exhibits spatial locality.

thrashing—Excessive paging activity causing low processor utilization that occurs when a process's memory allocation is smaller than its working set. This results in poor performance, as the process spends most of its time waiting as pages are transferred between secondary storage and main memory.

voluntary page release—Occurrence when a process explicitly releases a page frame that it no longer needs. This can improve performance by reducing the number of unused page frames allocated to a process, leaving more memory available.

working set—A program's favored subset of pages in main memory. Given a working set window, w , the process's working set of pages $W(t, w)$, is defined as the set of pages it references during the process-time interval $t - w$ to t .

working set theory of program behavior—Theory presented by Denning; which asserts that for a program to run efficiently, the system must maintain that program's favored subset (i.e., its working set) of pages in main memory. Given a working set window, w , the process's working set of pages $W(t, w)$, is defined as the set of pages referenced by the process during the process-time interval $t - w$ to t . Choosing the window size, w , is a crucial aspect of implementing working set memory management.

working set window size—Value that determines how far into the past the system should consider to determine what pages are in the process's working set.

Exercises

11.1 Discuss the goals of each of the following memory management strategies in the context of virtual memory systems with paging.

- fetch strategy
- placement strategy
- replacement strategy

11.2 Explain why memory management in pure segmentation systems is quite similar to memory management in variable-partition multiprogramming systems.

11.3 One particular virtual memory computer system with combined segmentation and paging supports a degree of multiprogramming of 10. Instruction pages (reentrant code) are maintained separately from data pages (which are modifiable).

514 Virtual Memory Management

You have studied the system in operation and have made the following observations: (1) Most procedure segments are many pages long and (2) most data segments use only a small fraction of a page.

Your associate has proposed that one way to get better utilization out of memory is to pack several of each user's data segments onto an individual page. Comment on this proposal considering issues of:

- a. memory use
- b. execution efficiency
- c. protection
- d. sharing

11.4 Today there is much interest in anticipatory paging and in anticipatory resource allocation in general. What useful information might each of the following supply to an anticipatory paging mechanism?

- a. the programmer
- b. the compiler
- c. the operating system
- d. a log of past executions of the program

11.5 It is known that in the general case we cannot predict the path of execution of an arbitrary program. If we could, then we would be able to solve the Halting Problem—which is known to be unsolvable. Explain the ramifications for the effectiveness of anticipatory resource-allocation mechanisms.

11.6 Suppose a memory manager has narrowed its page-replacement decision to one of two pages. Suppose that one of the pages is shared by several processes, and the other is in use by only a single process. Should the memory manager always replace the nonshared page? Explain.

11.7 Discuss each of the following unconventional (if not whimsical) page-replacement schemes in the context of a virtual memory multiprogramming system servicing both batch and interactive processes.

- a. "Global LIFO"—The page brought into main memory most recently is replaced.
- b. "Local LIFO"—The page brought in most recently by the process which requested the incoming page is replaced.
- c. "Tired Page"—The most heavily referenced page in the system is replaced.
- d. "Battered Page"—The most heavily modified page in the system is replaced. One variant would count the number of writes to the page. Another would consider the percentage of the page that has been modified.

11.8 Suppose a memory manager decides which page to replace solely on the basis of examining the referenced and modified bits for each page frame. List several incorrect decisions the memory manager might make.

11.9 List several reasons why it is necessary to prevent certain pages from being paged out of main memory.

11.10 Why is it generally more desirable to replace an unmodified page rather than a modified one? In what circumstances might it be more desirable to replace a modified page?

11.11 For each of the following pairs of replacement strategies, explain how a page-reference sequence could result in both strategies choosing (1) the same page for replacement and (2) a different page for replacement.

- a. LRU,NUR
- b. LRU,LFU
- c. LRU, FIFO
- d. NUR,LFU
- e. LFU,FIFO
- f. NUR,FIFO.

11.12 The optimal (MIN) page-replacement strategy is unrealizable because the future cannot be predicted. There are circumstances, however, in which MIN can be implemented. What are they?

11.13 Suppose a memory manager has chosen a modified page for replacement. This page must be sent to secondary storage before the new page may be placed in its page frame. Therefore, the memory manager requests an I/O operation to write this page to secondary storage. An entry is made in a list of I/O requests waiting to be serviced. Thus the page will remain in main memory for some time before the requested I/O operation is performed. Now suppose that, while the other I/O requests are serviced, a running process requests the page to be replaced. How should the memory manager react?

11.14 Design an experiment for a paged system to demonstrate the phenomena of temporal and spatial locality.

11.15 A programmer who writes programs specifically to exhibit good locality can expect marked improvement in their execution efficiency. List several strategies a programmer can use to improve locality. In particular, what high-level language features should be emphasized?

11.16 Suppose the bus between memory and secondary storage is experiencing heavy page traffic. Does this imply thrashing? Explain.

11.17 Why might a global page-replacement policy (in which an incoming page may replace a page of any process) be more susceptible to thrashing than a local page-replacement policy

(in which an incoming page may replace only a page belonging to the same process)?

11.18 Discuss the trade-offs between giving each process more page frames than it needs (to prevent thrashing) and the resulting fragmentation of main memory.

11.19 Suggest a heuristic that a memory manager might use to determine if main memory has become overcommitted.

11.20 The working set of a process may be defined in several ways. Discuss the merits of each of the following schemes for deciding which pages constitute a process's working set.

- those pages that the process has referenced in the last w seconds of wall-clock time
- those pages that the process has referenced in the last w seconds of virtual time (i.e., time during which the process was running on a processor)
- the last k different pages referenced by the process
- those pages on which a process made its last r instruction or data references
- those pages that the process has referenced in the last w seconds of virtual time with a frequency greater than/times per virtual second.

11.21 Give an example in which a working set page-replacement strategy would replace:

- the best possible page
- the worst possible page

11.22 One difficulty in implementing a working set memory management strategy is that when a process requests a new page, it is difficult to determine whether that process is transitioning to a new working set or is expanding its current working set. In the first case it is better for the memory manager to replace one of the process's pages; in the latter it is better for the memory manager to increase the process's page frame allocation. How might a memory manager decide which case is appropriate?

11.23 Suppose all active processes have established their working sets in main memory. As localities change, working sets could grow and main memory could become overcommitted, causing thrashing. Discuss the relative merits of each of the following preventive strategies for preventing this.

- Never initiate a new process if main memory is already 80 percent or more committed.
- Never initiate a new process if main memory is already 97 percent or more committed.
- When a new process is initiated, assign it a maximum working set size beyond which it will not be allowed to grow.

11.24 The interaction between the various components of an operating system is critical to achieving good performance. Discuss the interaction between the memory manager and the job initiator (i.e., admission scheduler) in a virtual-memory multiprogramming system. In particular, suppose the memory manager uses a working set memory-management approach.

11.25 Consider the following experiment and explain the observations.

A process is run by itself on a paged machine. It begins execution with its first procedure page. As it runs, the pages it needs are demand-paged into available page frames. The number of available page frames far exceeds the number of pages in the process. But there is a dial external to the computer that allows a person to set the maximum number of page frames the process may use.

Initially, the dial is set at two frames, and the program is run to completion. The dial is then set at three frames, and again the program is run to completion. This continues until the dial is eventually set to the number of available page frames in main memory, and the process is run for the last time. For each run, the run time of the process is recorded.

Observations:

As the dial is changed from two to three to four, the run times improve dramatically. From four to five to six, the run times still improve each time, but less dramatically. With the dial settings of seven and higher, the run time remains essentially constant.

11.26 An operating systems designer has proposed the "PD memory management strategy" that operates as follows. Exactly two page frames are allocated to each active process. These frames hold the most recently referenced procedure page, called the P-page, and the most recently referenced data page, called the D-page. When a page fault occurs, if the referenced page is a procedure page, the strategy replaces the P-page, and if the referenced page is a data page, the strategy replaces the D-page.

The designer says that the chief virtue of the scheme is that it trivializes all aspects of memory management and thus has low overhead.

- How are each of the following memory management strategies handled under the PD scheme?
 - fetch
 - placement
 - replacement
- In what circumstances would PD actually yield better results than working set memory management?
- In what circumstances would PD yield poor results?

11.27 Summarize the arguments for and against (1) small page sizes and (2) large page sizes.

516 Virtual Memory Management

n.28 The Multics system was originally designed to manage 64-word pages and 1024-word pages (the dual-page-size scheme was eventually abandoned).

- a. What factors do you suppose motivated this design decision?
- b. What effect does this dual-page-size approach have on memory management strategies?

11.29 Discuss the use of each of the following hardware features in virtual memory systems.

- a. dynamic address mapping mechanisms
- b. associative memory
- c. direct-mapped cache memory
- d. "page-referenced" bit
- e. "page-modified" bit
- f. "page-in-transit" bit (signifying that a page is currently being input to a particular page frame)

11.30 If programs are carefully organized in a paged system so that references are highly localized to small groups of pages, the resulting performance improvements can be impressive. Most programs today are being written in high-level languages, however, so the programmer does not generally have access to information rich enough to aid in producing programs with good organization. Because the path of execution a program will take cannot be predicted, it is difficult to know precisely which sections of code will be used intensively.

One hope for improving program organization is called dynamic program restructuring. Here, the operating system monitors the execution characteristics of a program, rearranging the code and data so that the more active items are placed together on pages.

- a. What execution characteristics of a large program should be monitored to facilitate dynamic program restructuring?
- b. How would a dynamic program restructuring mechanism use this information to make effective restructuring decisions?

11.31 The key to the proper and efficient operation of the page-fault-frequency (PFF) page-replacement strategy is the selection of the threshold interfault time values. Answer each of the following questions.

- a. What are the consequences of selecting too large an upper value?
- b. What are the consequences of selecting too small a lower value?
- c. Should the values remain fixed, or should they be adjusted dynamically?
- d. What criteria would you use to adjust the values dynamically?

11.32 Compare and contrast the page-fault-frequency (PFF) page-replacement strategy with the working set (WS) page-replacement strategy. Be sure to consider each of the following:

- a. execution-time overhead
- b. memory overhead for holding the information necessary to support the strategy

11.33 A possible weakness of the second-chance page-replacement strategy is that if the list of pages is short, then an active page that has just moved to the back of the list after having had its referenced bit set off could quickly move to the front of the list and be selected for replacement before its referenced bit is set on. Comment on this phenomenon. Is it an anomaly in second-chance page replacement? Defend your answer carefully.

11.34 Suppose a system is not currently thrashing. List as many factors as you can that might cause it to begin thrashing. What measures should the operating system take, once thrashing is detected? Can thrashing be absolutely prevented? If so, at what cost? If not, explain why not.

11.35 A system receives a series of page references in the following order: 1, 1, 3, 5, 2, 2, 6, 8, 7, 6, 2, 1, 5, 5, 5, 1, 4, 9, 7, 7. The system has five page frames. If all of the frames are initially empty, calculate the number of page faults using each of these algorithms:

- a. FIFO
- b. LRU
- c. second chance
- d. OPT

11.36 Should a process be penalized for generating excessive page faults?

11.37 Consider a process experiencing a large number of page faults. Describe the effect, if any, of increasing this process's scheduling priority.

Suggested Projects

11.38 Some types of processes perform well under certain page-replacement strategies and poorly under others. Discuss the possibility of implementing a memory manager that would

dynamically determine a process's type and then select and use the appropriate page-replacement strategy for that process.

11.39 Suppose you are aware of the page-replacement strategy used by the multiuser timesharing system on which you are running. For each of the page-replacement strategies discussed in this chapter, what countermeasures might you take to force the operating system to give your process(es) favored treatment, when in fact the operating system might normally do otherwise? What measures might an operating system take to detect such processes and prevent them from obtaining such preferred treatment? Prepare a paper describing your solution.

Suggested Simulations

11.42 Develop a simulator program that will enable you to compare and contrast the operation of each of the page-replacement strategies discussed in this chapter. Your simulator needs to be concerned with transitions between pages in virtual space, but not with the instruction-by-instruction execution of programs. Assign a random but high probability to the event of remaining on the same page on which the previous instruction was executed. When page transitions occur, the probability of transferring to the next page (or the previous page) in virtual space should be higher than that of transferring to some remote page. Assume a moderately loaded system so that page-replacement decisions are common. Assume that the same replacement strategy applies to all processes on the system for a given run of the simulation. Your simulator should maintain statistics on the performance of each replacement strategy. Include features to enable you to fine-tune the simulator; i.e., you should be able to adjust the working set window size, the page-fault-frequency threshold values, and so on. When the simulator is completed, develop a new page-replacement strategy and compare the results to those of the other strategies in the text.

11.43 Develop a simulation program to investigate how changing the number of pages allocated to a process affects the number of page faults it experiences. Use reference strings that simulate spatial locality over both large and small regions of memory and temporal locality over both large and small periods. Also test the simulation using reference strings that are

essentially random. Describe the optimal page-replacement algorithm for each reference string. You might also wish to simulate the effect of multiprogramming by interleaving different reference patterns.

11.41 Prepare a research paper on hardware support for the working set model.

11.44 Develop a program that simulates page-fault-frequency (PFF) page replacement for a particular reference string. Determine the effect of changing the threshold values each time the program runs. Use reference strings that simulate spatial locality over both large and small regions of memory and temporal locality over both large and small periods. Also test the simulation using reference strings that are essentially random. Describe the optimal page-replacement algorithm for each reference string. Enhance your simulation so that it dynamically adjusts these values in response to the reference string it encounters.

11.45 Develop a program that compares page-fault-frequency (PFF) page replacement and working set (WS) page replacement for a particular reference string. Determine the effect of changing the threshold values for PFF and window size for WS each time the program runs. Use reference strings that simulate spatial locality over both large and small regions of memory and temporal locality over both large and small periods. Also test the simulation using reference strings that are essentially random. Describe the optimal page-replacement algorithm for each reference string.

Recommended Reading

Owing to its wide application in operating system design, virtual memory management is a rich field. Seminal papers, such as Denning's discussion of the working set model⁸⁸ and his summary of virtual memory concepts,⁸⁹ offer a clear presentation of the concerns in virtual memory management, most of which are still valid today. Fetch policies were discussed by Aho et al.⁹⁰ and further analyzed by Kaplan et al.⁹¹

Optimal page replacement was discussed by Belady,⁹² who also discovered the FIFO anomaly.⁹³ Though common

algorithms such as FIFO and approximations to LRU can be found as pieces of some memory management algorithms, typically they are not used alone. More sophisticated algorithms involving graphs and coloring have been developed in the literature.⁹⁴⁻⁹⁵ Morris discusses the use of custom hardware to make working set storage management more efficient.⁹⁶

The question of the most appropriate page size has been studied in the literature.⁹⁷⁻⁹⁸ To meet different application needs, modern general-purpose processors have begun to sup-

port multiple page sizes.¹⁰⁰ Program behavior under paging continues to be the subject of many studies that seek to increase system throughput and performance by reducing the

number of page faults.^{101, 102, 103, 104, 105, 106, 107, 108} The bibliography for this chapter is located on our Web site at www.ditel.com/books/os3e/Bibliography.pdf.

Works Cited

- Denning, P. J., "The Working Set Model for Program Behavior," *Communications of the ACM*, Vol. 11, No. 5, May 1968, pp. 323-333.
- Belady, L. A., "A Study of Replacement Algorithms for Virtual Storage Computers," *IBM Systems Journal*, Vol. 5, No. 2, 1966, pp. 78-101.
- Baer, X, and G. R. Sager, "Dynamic Improvement of Locality in Virtual Memory Systems," *IEEE Transactions on Software Engineering*, Vol. SE-1, March 1976, pp. 54-62.
- Aho, A. V; P. J. Denning; and J. D. Ullman, "Principles of Optimal Page Replacement," *Journal of the ACM*, Vol. 18, No. 1, January 1971, pp. 80-93.
- Minsky, M. L., *Computation: Finite and Infinite Machines*, Englewood Cliffs, NJ: Prentice-Hall, 1967.
- Hennie, E, *Introduction to Computability*, Reading, MA: Addison-Wesley, 1977.
- Buzen, X P., "Fundamental Laws of Computer System Performance," *Proceedings of the 1976 ACM SIGMETRICS Conference on Computer Performance Modeling Measurement and Evaluation*, 1976, pp. 200-210.
- Randell, B., and C. X. Kuehner, "Dynamic Storage Allocation Systems," *CACM*, Vol. 11, No. 5, May 1968, pp. 297-306.
- Lorin, H., and H. Deitel, *Operating Systems*, Reading, MA: Addison-Wesley, 1981.
- Trivedi, K. S., "Prepaging and Applications to Array Algorithms," *IEEE Transactions on Computers*, Vol. C-25, September 1976, pp. 915-921.
- Smith, A. X., "Sequential Program Prefetching in Memory Hierarchies," *Computer*, Vol. 11, No. 12, December 1978, pp. 7-21.
- Trivedi, K. S., "Prepaging and Applications to Array Algorithms," *IEEE Transactions on Computers*, Vol. C-25, September 1976, pp. 915-921.
- Kaplan, S., et al, "Adaptive Caching for Demand Repaging," *Proceedings of the Third International Symposium on Memory Management*, 2002, pp. 114-116.
- Linux Source, <lxr.linux.no/source/mm/memory.c?v=2.5.56>, line 1010 (swpin_readahead).
- Linux Source, <lxr.linux.no/source/mm/swap.c?v=2.6.0-test2>, line 100 (swap_setup).
- Belady, L. A., "A Study of Replacement Algorithms for Virtual Storage Computers," *IBM Systems Journal*, Vol. 5, No. 2, 1966, pp. 78-101.
- Denning, P. X., "Virtual Memory," *ACM Computing Surveys*, Vol. 2, No. 3, September 1970, pp. 153-189.
- Mattson, R. L.; J. Gecsie; D. R. Slutz; and I. L. Traiger, "Evaluation Techniques for Storage Hierarchies," *IBM Systems Journal*, Vol. 9, No. 2, 1970, pp. 78-117.
- Prieve, B. G, and R. S. Fabry, "VMIN-An Optimal Variable Space Page Replacement Algorithm," *Communications of the ACM*, Vol. 19, No. 5, May 1976, pp. 295-297.
- Budzinski, R.; E. Davidson; W. Mayeda; and H. Stone, "DMIN: An Algorithm for Computing the Optimal Dynamic Allocation in a Virtual Memory Computer," *IEEE Transactions on Software Engineering*, Vol. SE-7, No. 1, January 1981, pp. 113-121.
- Belady, L. A., and C. J. Kuehner, "Dynamic Space Sharing in Computer Systems," *Communications of the ACM*, Vol. 12, No. 5, May 1969, pp. 282-288.
- Turner, R., and H. Levy, "Segmented FIFO Page Replacement," *Proceedings of the 1981 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*, 1981, pp. 48-51.
- Babaoglu, O., and W. Joy, "Converting a Swap-Based System to Do Paging in an Architecture Lacking Page-Referenced Bits," *Proceedings of the Eighth Symposium on Operating Systems Principles, ACM*, Vol. 15, No. 5, December 1981, pp. 78-86.
- Carr, R. W., *Virtual Memory Management*. UMI Research Press 1984.
- Borodin, A.; S. Irani; P. Raghavan; and B. Schieber, "Competitive Paging with Locality of Reference," in *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing*, New Orleans, Louisiana, May 1991, pp. 249-259.
- Irani, S.; A. Karlin; and S. Phillips, "Strongly Competitive Algorithms for Paging with Locality of Reference," *Proceedings of the Third Annual ACM-SIAM Symposium on Discrete Algorithms*, 1992, pp. 228-236.
- Albers, S.; L. Favrholdt; and O. Giel, "On Paging with Locality of Reference," *Proceedings of the Thirty-Fourth Annual ACM Symposium on Theory of Computation*, 2002, pp. 258-267.
- Fiat, A., and Z. Rosen, "Experimental Studies of Access Graph Based Heuristics: Beating the LRU Standard?" in *Proceedings of the Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, New Orleans, Louisiana, January 1997, pp. 63-72.
- Fiat, A., and Z. Rosen, "Experimental Studies of Access Graph Based Heuristics: Beating the LRU Standard?" In *Proceedings of the Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, New Orleans, Louisiana, January 1997, pp. 63-72.
- Denning, P. J., "The Working Set Model for Program Behavior," *Communications of the ACM*, Vol. 11, No. 5, May 1968, pp. 323-333.

31. Denning, P. X, "Resource Allocation in Multiprocess Computer Systems," Ph.D. Thesis, Report MAC-TR-50 M.I.T. Project MAC, May 1968.
32. Hatfield, D, "Experiments on Page Size, Program Access Patterns, and Virtual Memory Performance," *IBM Journal of Research and Development*, Vol. 15, No. 1, January 1972, pp. 58-62.
33. Denning, P. X, "The Working Set Model for Program Behavior," *Communications of the ACM*, Vol. 11, No. 5, May 1968, pp. 323-333.
34. Denning, P. X, "Working Sets Past and Present," *IEEE Transactions on Software Engineering*, Vol. SE-6, No. 1, January 1980, pp. 64-84.
35. Denning, P. X, "Thrashing: Its Causes and Preventions," *AFIPS Conference Proceedings*, Vol. 33, 1968 FJCC, pp. 915-922.
36. Rodriguez-Rosell, X, "Empirical Working Set Behavior," *Communications of the ACM*, Vol. 16, No. 9, 1973, pp. 556-560.
37. Fogel, M., "The VMOS Paging Algorithm: A Practical Implementation of the Working Set Model," *Operating Systems Review*, Vol. 8, No. 1, January 1974, pp. 8-16.
38. Levy, H. M., and P. H. Lipman, "Virtual Memory Management in the VAX/VMS Operating System," *Computer*, Vol. 15, No. 3, March 1982, pp. 35-41.
39. Kenah, L. X; R. E. Goldenberg; and S. F. Bate, *VAXNMS Internals and Data Structures*, Bedford, MA: Digital Press, 1988.
40. Bryant, P., "Predicting Working Set Sizes," *IBM Journal of Research and Development*, Vol. 19, No. 3, May 1975, pp. 221-229.
41. Morris, X B., "Demand Paging through the Use of Working Sets on the Maniac II," *Communications of the ACM*, Vol. 15, No. 10, October 1972, pp. 867-872.
42. Chu, W W, and H. Opderbeck, "The Page Fault Frequency Replacement Algorithm," *Proceedings AFIPS Fall Joint Computer Conference*, Vol. 41, No. 1, 1972, pp. 597-609.
43. Opderbeck, H., and W. W. Chu, "Performance of the Page Fault Frequency Algorithm in a Multiprogramming Environment," *Proceedings of FIP Congress, 191 A*, pp. 235-241.
44. Sadeh, E., "An Analysis of the Performance of the Page Fault Frequency (PFF) Replacement Algorithm," *Proceedings of the Fifth ACM Symposium on Operating Systems Principles*, November 1975, pp. 6-13.
45. Chu, W W., and H. Opderbeck, "Program Behavior and the Page-Fault-Frequency Replacement Algorithm," *Computer*, Vol. 9, No. 11, November 1976, pp. 29-38.
46. Gupta, R. K., and M. A. Franklin, "Working Set and Page Fault Frequency Replacement Algorithms: A Performance Comparison," *IEEE Transactions on Computers*, Vol. C-27, August 1978, pp. 706-712.
47. Ganapathy, N, and C. Schimmel, "General Purpose Operating System Support for Multiple Page Sizes," *Proceedings of the USENIX Conference, 1998*.
48. Batson, A. P.; S. Ju; and D. Wood, "Measurements of Segment Size," *Communications of the ACM*, Vol. 13, No. 3, March 1970, pp. 155-159.
49. Chu, W. W., and H. Opderbeck, "Performance of Replacement Algorithms with Different Page Sizes," *Computer*, Vol. 7, No. 11, November 1974, pp. 14-21.
50. Denning, P. X, "Working Sets Past and Present," *IEEE Transactions on Software Engineering*, Vol. SE-6, No. 1, January 1980, pp. 64-84.
51. Ganapathy, N, and C. Schimmel, "General Purpose Operating System Support for Multiple Page Sizes," *Proceedings of the USENIX Conference, 1998*.
52. Talluri, M.; S. Kong; M. D. Hill; and D. A. Patterson, "Tradeoffs in Supporting Two Page Sizes," in *Proceedings of the 19th International Symposium on Computer Architecture*, Gold Coast, Australia, May 1992, pp. 415-424.
53. Talluri, M.; S. Kong; M. D. Hill; and D. A. Patterson, "Tradeoffs in Supporting Two Page Sizes," in *Proceedings of the 19th International Symposium on Computer Architecture*, Gold Coast, Australia, May 1992, pp. 415-424.
54. McNamee, D., "Flexible Physical Memory Management," Department of Computer Science and Engineering, University of Washington, September 1995.
55. Talluri, M.; S. Kong; M. D. Hill; and D. A. Patterson, "Tradeoffs in Supporting Two Page Sizes," in *Proceedings of the 19th International Symposium on Computer Architecture*, Gold Coast, Australia, May 1992, pp. 415-424.
56. *IA-32 Intel Architecture Software Developer's Manual*, Vol. 3, *System Programmer's Guide*, Intel, 2002, p. 3-19.
57. "UltraSPARC II Detailed View," Modified July 29, 2003, <www.sun.com/processors/intraSPARC-II/details.html>.
58. *PowerPC Microprocessor Family: Programming Environments Manual for 64 and 32-Bit Microprocessors*, Version 2.0, IBM, June 10, 2003, pp. 258, 282.
59. Belady, L. A., "A Study of Replacement Algorithms for Virtual Storage Computers," *IBM Systems Journal*, Vol. 5, No. 2, 1966, pp. 78-101.
60. Fine, E. G; C. W Jackson; and P. V McIsaac, "Dynamic Program Behavior under Paging," *ACM 21st National Conference Proceedings*, 1966, pp. 223-228.
61. Coffman, E. G, Jr., and L. C. Varian, "Further Experimental Data on the Behavior of Programs in a Paging Environment," *Communications of the ACM*, Vol. 11, No. 7, July 1968, pp. 471-474.
62. Freibergs, I. F., "The Dynamic Behavior of Programs," *Proceedings AFIPS Fall Joint Computer Conference*, Vol. 33, Part 2, 1968, pp. 1163-1167.
63. Hatfield, D., "Experiments on Page Size, Program Access Patterns, and Virtual Memory Performance," *IBM Journal of Research and Development*, Vol. 15, No. 1, January 1972, pp. 58-62.
64. Freibergs, I. F., "The Dynamic Behavior of Programs," *Proceedings AFIPS Fall Joint Computer Conference*, Vol. 33, Part 2, 1968, pp. 1163-1167.

65. Spirn, J. R., and P. J. Denning, "Experiments with Program Locality," *AFIPS Conference Proceedings*, Vol. 41, 1972 FJCC, pp. 611-621.
66. Morrison, J. E., "User Program Performance in Virtual Storage Systems," *IBM Systems Journal*, Vol. 12, No. 3, 1973, pp. 216-237.
67. Rodriguez-Rosell, J. "Empirical Working Set Behavior," *Communications of the ACM*, Vol. 16, No. 9, 1973, pp. 556-560.
68. Chu, W. W., and H. Opderbeck, "Performance of Replacement Algorithms with Different Page Sizes," *Computer*, Vol. 7, No. 11, November 1974, pp. 14-21.
69. Oliver, N. A., "Experimental Data on Page Replacement Algorithms," *Proceedings of AFIPS*, 1974 NCC 43, Montvale, NJ: AFIPS Press, 1974, pp. 179-184.
70. Opderbeck, H., and W. W. Chu, "Performance of the Page Fault Frequency Algorithm in a Multiprogramming Environment," *Proceedings of FIP Congress*, 191 A, pp. 235-241.
71. Sadeh, E., "An Analysis of the Performance of the Page Fault Frequency (PFF) Replacement Algorithm," *Proceedings of the Fifth ACM Symposium on Operating Systems Principles*, November 1975, pp. 6-13.
72. Baer, I. and G. R. Sager, "Dynamic Improvement of Locality in Virtual Memory Systems," *IEEE Transactions on Software Engineering*, SE-1, March 1976, pp. 54-62.
73. Potier, D., "Analysis of Demand Paging Policies with Swapped Working Sets," *Proceedings of the Sixth ACM Symposium on Operating Systems Principles*, November 1977, pp. 125-131.
74. Franklin, M. A.; G. S. Graham; and R. K. Gupta, "Anomalies with Variable Partition Paging Algorithms," *Communications of the ACM*, Vol. 21, No. 3, March 1978, pp. 232-236.
75. Gupta, R. K., and M. A. Franklin, "Working Set and Page Fault Frequency Replacement Algorithms: A Performance Comparison," *IEEE Transactions on Computers*, Vol. C-27, August 1978, pp. 706-712.
76. Denning, P. I., "Working Sets Past and Present," *IEEE Transactions on Software Engineering*, Vol. SE-6, No. 1, January 1980, pp. 64-84.
77. Irani, S.; A. Karlin; and S. Phillips, "Strongly Competitive Algorithms for Paging with Locality of Reference," *Proceedings of the Third Annual ACM-SIAM Symposium on Discrete Algorithms*, 1992, pp. 228-236.
78. Glass, G. and Pei Cao, "Adaptive Page Replacement Based on Memory Reference Behavior," *Proceedings of the 1997 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pp. 115-126, June 15-18, 1997, Seattle, Washington.
79. Ganapathy, N. and C. Schimmel, "General Purpose Operating System Support for Multiple Page Sizes," *Proceedings of the USENIX Conference*, 1998.
80. Albers, S.; L. Favrholdt; and O. Giel, "On Paging with Locality of Reference," *Proceedings of the 34th Annual ACM Symposium on Theory of Computation*, 2002, pp. 258-267.
81. Franklin, M. A.; G. S. Graham; and R. K. Gupta, "Anomalies with Variable Partition Paging Algorithms," *Communications of the ACM*, Vol. 21, No. 3, March 1978, pp. 232-236.
82. Glass, G., and Pei Cao, "Adaptive Page Replacement Based on Memory Reference Behavior," *Proceedings of the 1997 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pp. 115-126, June 15-18, 1997, Seattle, Washington, United States.
83. Glass, G. and Pei Cao, "Adaptive Page Replacement Based on Memory Reference Behavior," *Proceedings of the 1997 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pp. 115-126, June 15-18, 1997, Seattle, Washington, United States.
84. A. Arcangeli, "Le novita' nel Kernel Linux," December 7, 2001. <old.lwn.net/2001/1213/aa-vm-talk/mgp00001.html>.
85. A. Arcangeli, "Le novita' nel Kernel Linux," December 7, 2001. <old.lwn.net/2001/1213/aa-vm-talk/mgp00001.html>.
86. Linux kernel source code version 2.5.75, <www.kernel.org>.
87. Linux kernel source code version 2.5.75, <www.kernel.org>.
88. Denning, P. J., "The Working Set Model for Program Behavior," *Communications of the ACM*, Vol. 11, No. 5, May 1968, pp. 323-333.
89. Denning, P. J., "Virtual Memory," *ACM Computing Surveys*, Vol. 2, No. 3, September 1970, pp. 153-189.
90. Aho, A. V.; P. J. Denning; and J. D. Ullman, "Principles of Optimal Page Replacement," *Journal of the ACM*, Vol. 18, No. 1, January 1971, pp. 80-93.
91. Kaplan, S., et al., "Adaptive Caching for Demand Prepaging," *Proceedings of the Third International Symposium on Memory Management*, 2002, pp. 114-116.
92. Belady, L. A., "A Study of Replacement Algorithms for Virtual Storage Computers," *IBM Systems Journal*, Vol. 5, No. 2, 1960, pp. 78-101.
93. Belady, L. A., and C. J. Kuehner, "Dynamic Space Sharing in Computer Systems," *Communications of the ACM*, Vol. 12, No. 5, May 1969, pp. 282-288.
94. Irani, S.; A. Karlin; and S. Phillips, "Strongly Competitive Algorithms for Paging with Locality of Reference," *Proceedings of the Third Annual ACM-SIAM Symposium on Discrete Algorithms*, 1992, pp. 228-236.
95. Albers, S.; L. Favrholdt; and O. Giel, "On Paging with Locality of Reference," *Proceedings of the 34th Annual ACM Symposium on Theory of Computation*, 2002, pp. 258-267.
96. Morris, J. B., "Demand Paging through the Use of Working Set on the Maniac II," *Communications of the ACM*, Vol. 15, No. 10, October 1972, pp. 867-872.
97. Batson, A. P.; S. Ju; and D. Wood, "Measurements of Segment Size," *Communications of the ACM*, Vol. 13, No. 3, March 1970, pp. 155-159.
98. Chu, W. W., and H. Opderbeck, "Performance of Replacement Algorithms with Different Page Sizes," *Computer*, Vol. 7, No. 11, November 1974, pp. 14-21.

Works Cited 521

99. Denning, P. J., "Working Sets Past and Present," *IEEE Transactions on Software Engineering*, Vol. SE-6, No. 1, January 1980, pp. 64-84.
100. Ganapathy, N., and C. Schimmel, "General Purpose Operating System Support for Multiple Page Sizes," *Proceedings of the USENIX Conference*, 1998.
101. Freibergs, I. E., "The Dynamic Behavior of Programs," *Proceedings AFIPS Fall Joint Computer Conference*, Vol. 33, Part 2, 1968, pp. 1163-1167.
102. Hatfield, D., "Experiments on Page Size, Program Access Patterns, and Virtual Memory Performance," *IBM Journal of Research and Development*, Vol. 15, No. 1, January 1972, pp. 58-62.
103. Morrison, J. E., "User Program Performance in Virtual Storage Systems," *IBM Systems Journal*, Vol. 12, No. 3, 1973, pp. 216-237.
104. Sadeh, E., "An Analysis of the Performance of the Page Fault Frequency (PFF) Replacement Algorithm," *Proceedings of the Fifth ACM Symposium on Operating Systems Principles*, November 1975, pp. 6-13.
105. Gupta, R. K., and M. A. Franklin, "Working Set and Page Fault Frequency Replacement Algorithms: A Performance Comparison," *IEEE Transactions on Computers*, Vol. C-27, August 1978, pp. 706-712.
106. Denning, P. J., "Working Sets Past and Present," *IEEE Transactions on Software Engineering*, Vol. SE-6, No. 1, January 1980, pp. 64-84.
107. Irani, S.; A. Karlin; and S. Phillips, "Strongly Competitive Algorithms for Paging with Locality of Reference," *Proceedings of the Third Annual ACM-SIAM Symposium on Discrete Algorithms*, 1992, pp. 228-236.
108. Albers, S.; L. Favrholt; and O. Giel, "On Paging with Locality of Reference," *Proceedings of the 34th Annual ACM Symposium on Theory of Computation*, 2002, pp. 258-267.

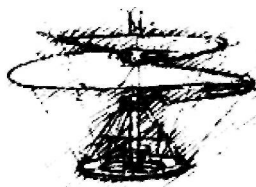
Secondary Storage, Files and Databases

A fair request should be followed by the deed in silence.

—Dante—

Part 4

Computers store programs and vast amounts of data as files and databases on secondary storage devices. The following two chapters explain how operating systems organize and manage data on these devices. We explain the operation of the enormously popular moving-head disk storage device and show how to achieve maximum performance with seek and rotational optimization strategies. RAID (Redundant Arrays of Independent Disks) systems, which achieve high levels of performance and fault tolerance, are presented. We discuss file systems and examine how files are allocated on disk, how free space is managed and how file data is accessed and protected. File servers, and how they are used in distributed systems, are explained. We introduce database systems, and discuss relational databases and the kinds of operating systems services that support database systems.



'Tis in my memory lock'd, And you yourself shall keep the key of it.

—William Shakespeare—

The path of duty lies in what is near, and man seeks for it in what is remote.

—Mencius—

A fair request should be followed by the deed in silence.

—Dante—

... the latter, in search of the hard latent value with which it alone is concerned, ~~sifts~~ sifts round the mass as instinctively and unerringly as a dog suspicious of some buried bone.

—William James—

Go where we will on the surface of things, men have been there before us.

—Henry David Thoreau—

The wheel that squeaks the loudest is the one that gets the grease.

—Josh Billings (Henry Wheeler Shaw) —

Chapter 12

Disk Performance Optimization

Objectives

After reading this chapter, you should understand:

- *how disk input/output is accomplished.*
- *the importance of optimizing disk performance.*
- *seek optimization and rotational optimization.*
- *various disk scheduling strategies.*
- *caching and buffering.*
- *other disk performance improvement techniques.*
- *key schemes for implementing redundant arrays of independent disks (RAID).*

Chapter Outline

12.1 *Introduction*

12.2 *Evolution of Secondary Storage*

12.3 *Characteristics of Moving-Head Disk Storage*

12.4 *Why Disk Scheduling Is Necessary*

12.5 *Disk Scheduling Strategies*

- 12.5.1 First-Come-First-Served (FCFS)
Disk Scheduling*
- 12.5.2 Shortest-Seek-Time-First (SSTF)
Disk Scheduling*
- Anecdote: Every Problem Has a Solution
and Every Solution Has a Problem*
- 12.5.3 SCAN Disk Scheduling*
- 12.5.4 C-SCAN Disk Scheduling*
- 12.5.5 FSCAN and N-Step SCAN Disk
Scheduling*
- 12.5.6 LOCK and C-LOCK Disk
Scheduling*

12.6 *Rotational Optimization*

- 12.6.1 SLTF Scheduling*
- 12.6.2 SPTF and SATF Scheduling*

12.7 *System Considerations*

*Operating Systems Thinking:
Saturation and Bottlenecks*

12.8 *Caching and Buffering*

12.9 *Other Disk Performance Techniques*

*Operating Systems Thinking:
Compression and Decompression*

*Operating Systems Thinking:
Redundancy*

12.10 *Redundant Arrays of Independent Disk (RAID)*

12.10.1 RAID Overview

*Operating Systems Thinking: Mission-
Critical Systems*

*Operating Systems Thinking: Fault
Tolerance*

- 12.10.2 Level 0 (Striping)*
- 12.10.3 Level 1 (Mirroring)*
- 12.10.4 Level 2 (Bit-Level Hamming ECC Parity)*
- 12.10.5 Level 3 (Bit-level XOR ECC Parity)*
- 12.10.6 Level 4 (Block-level XOR ECC Parity)*
- 12.10.7 Level 5 (Block-Level Distributed XOR ECC Parity)*

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

12.1 Introduction

In recent years, processor and main memory speeds have increased more rapidly than those of secondary storage devices, such as hard disks. As a result, processes requesting data on secondary storage tend to experience relatively long service delays. In this chapter, we discuss the characteristics of moving-head disk storage and consider how operating system designers can manage such devices to provide better service to processes. We explain how to optimize disk performance by reordering disk requests to increase throughput, decrease response times and reduce the Variance of response times. We also discuss how operating systems reorganize data on disk and exploit buffers and caches to boost performance. Finally, we discuss Redundant Arrays of Independent Disks (RAIDs), which improve disk access times and fault tolerance by servicing requests using multiple disks at once.

12.2 Evolution of Secondary Storage

In early computers, persistent data was stored on punched cards and punched paper tape, which used the absence or presence of holes to represent bits of data.¹ Writing software and loading it into the computer using such media was both labor intensive and slow. The need for an inexpensive, rewritable persistent storage device led researchers to develop magnetic storage, which records bits of data by changing the direction of magnetization of regions on the medium's surface. To access data, a current-carrying device called a **read-write head** hovers above the medium as it moves. The head reads data by measuring how the magnetized medium changes the current; it "writes" data by using the current to change the magnetization on the medium. A challenge in building these devices is that the head must float extremely close to the medium's surface without touching it.

In 1951, the designers of **UNIVAC 1 (UNIVersal Automatic Computer)** introduced **magnetic tape storage**, which was both persistent and rewritable.² Magnetic tape is a form of sequential access storage, like audio or video cassettes. Such a medium is inappropriate for transaction-processing applications, where, for example, the system must be able to locate and update any record in a fraction of a second. To address this problem, IBM introduced the first commercial **hard disk drive**, the **RAMAC (Random Access Method of Accounting and Control)**, in 1957. Hard disk drives are random access (also called direct access) devices because they are not limited to accessing data sequentially. RAMAC's capacity totalled five megabytes, and its cost was \$50,000; it was generally rented to installations for hundreds of dollars per month.³⁴ Although hard disks provided better performance than magnetic tape storage, their high cost limited their use to large installations.

As the decades passed, hard drive capacity and performance increased, while costs declined. Typical personal computer hard disk capacity increased from hundreds of megabytes to several gigabytes during the 1990s while prices fell to a few pennies per megabyte. By 2003, hard drive capacity had exceeded 200GB and cost less than a dollar per gigabyte.⁵ Due to mechanical constraints that we discuss in the

next section, hard disk speeds have improved more slowly than their capacity. As processor speeds increased and applications consumed larger amounts of data, systems became increasingly I/O bound.⁶

Research in persistent storage technology continues to focus on increasing capacity and performance. Some solutions attempt to improve the performance of existing magnetic disk devices; others employ novel techniques and media.

Self Review

1. Why have many of today's systems become I/O bound?
2. Why are disks more appropriate than tapes for secondary storage?

Ans: 1) Processor speeds have increased faster than hard disk speeds. 2) Magnetic tape performs well only in an application in which data is accessed sequentially. In transaction-processing applications and in multiprogrammed systems, requests to access secondary storage from multiple processes can lead to essentially random access patterns. In this case, direct access storage devices are essential.

12.3 Characteristics of Moving-Head Disk Storage

Unlike main memory, which provides (nearly) uniform access speed to all of its contents, moving-head disk storage exhibits variable access speed that depends on the relative positions of the read-write head and the requested data. Figure 12.1 shows a simplified view of a moving-head disk.⁷⁻⁹ Data is recorded on a series of

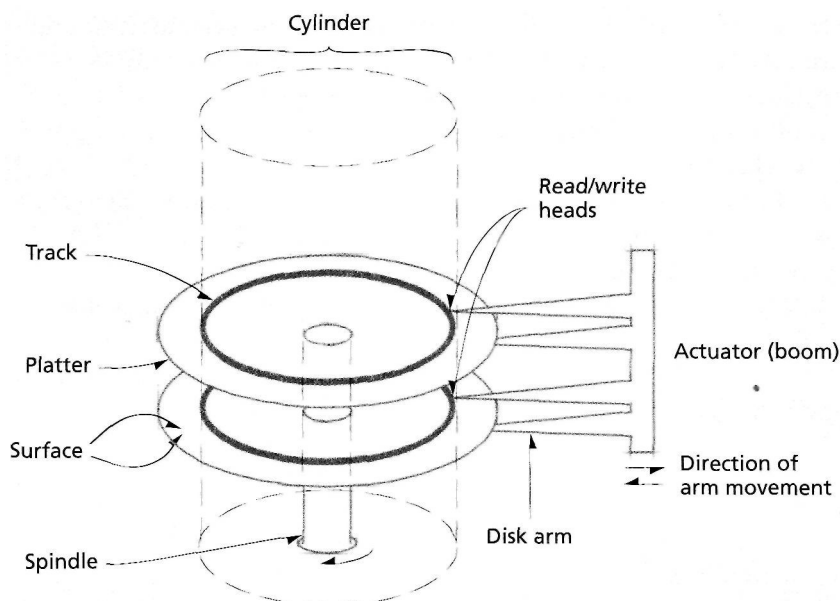


Figure 12.1 | Schematic side view of a moving-head disk.

magnetic disks, or **platters**, connected to a **spindle** that rotates at high speed (typically thousands of revolutions per minute¹⁰).

The data on each disk surface is accessed by a read-write head that is separated by a small amount of space from the surface (much smaller than a particle of smoke). For example, the disk in Fig. 12.1 contains two platters, each having two surfaces (top and bottom) and four read-write heads, one for each surface. A read-write head can access data immediately below (or above) it. Therefore, before data can be accessed, the portion of the disk surface at which the data is to be read (or written) must rotate until it is immediately below (or above) the read-write head. The time it takes for data to rotate from its current position to the beginning of the read-write head is called **rotational latency time**. A disk's average rotational latency is simply half the time it takes to complete one revolution. Most hard disks exhibit average rotational latency on the order of several milliseconds (Fig. 12.2).

As the platters spin, each read-write head sketches out a circular **track** of data on a disk surface. Each read-write head is positioned at the end of a **disk arm**, which is attached to an **actuator** (also called a **boom or moving-arm assembly**). The disk arm moves parallel to the surface of the disk. When the disk arm moves the read-write heads to a new position, a different vertical set of circular tracks, or **cylinder**, becomes accessible. The process of moving the disk arm to a new cylinder is called a **seek operation**.¹¹¹² To locate small units of data, disks divide tracks into several **sectors**, often 512 bytes (Fig. 12.3).¹³ Therefore, an operating system can locate a particular data item by specifying the head (which indicates which disk surface to read from), the cylinder (which indicates which track to read from), and the sector in which the data is located.

To access a particular record of data on a moving-head disk, several operations are usually necessary (Fig. 12.4). First, the disk arm must move to the appropriate cylinder (i.e., perform a **seek operation**). The time it takes for the head to move from its current cylinder to the one containing the data record is called the **seek time**. Then the portion of the disk on which the data record is stored must rotate until it is immediately under (or over) the read-write head. Then the record, which is of arbitrary size, must be made to spin by the read-write head. This is called **transmission time**. Because each of these operations involves mechanical movement, the total time a disk access takes is often an appreciable fraction of a second

<i>Model (Environment)</i>	<i>Average Seek Time (ms)</i>	<i>Average Rotational latency (ms)</i>
Maxtor DiamondMax Plus 9 (High-end desktop)	9.3	4.2
WD Caviar (High-end desktop)	8.9	4.2
Toshiba MK8025GAS (Laptop)	12.0	7.14
WD Raptor (Enterprise)	5.2	2.99
Cheetah 15K.3 (Enterprise)	3.6	2.0

Figure 12.2 | Hard disk track-to-track seek times and latency times.^{14, 15, 16, 17, 18}

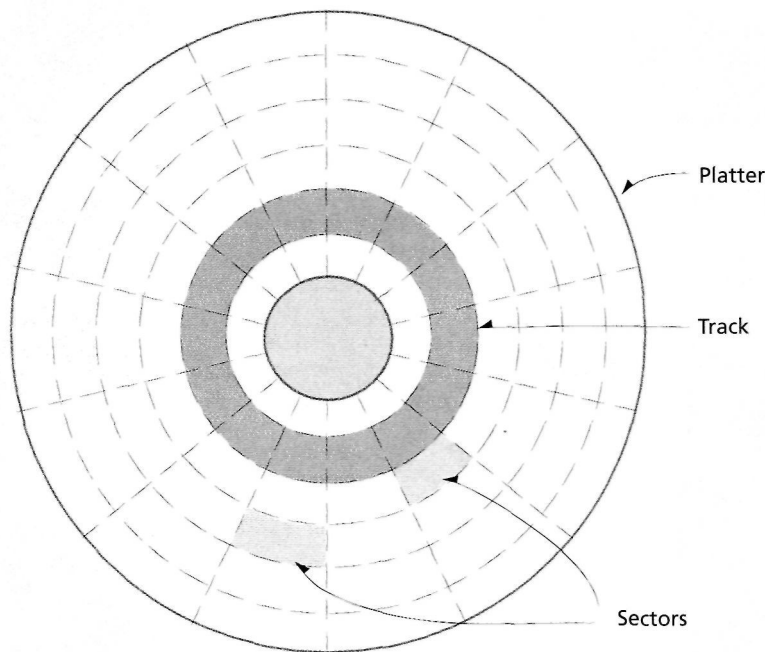


Figure 12.3 | Schematic top view of a disk surface.

(several milliseconds at least). During this same period of time, a processor can execute tens or even hundreds of millions of instructions.

Self Review

1. One focus of research in the field of storage technology is on increasing a hard disk's areal density (i.e., amount of data per unit area). How does increasing the areal density alone affect disk access times?
2. (T/F) Rotational latency is identical for every disk access.

Ans: 1) As the amount of data per unit area increases, then the amount of data contained in each track increases. If a particular data record is located on a single track, the transmission time decreases compared to that for a lower areal density because more data can be read per unit time. 2) False. The rotational latency depends on the location of the beginning of the requested sector relative to the position of the arm.

12.4 Why Disk Scheduling Is Necessary

Many processes can generate requests for reading and writing data on a disk simultaneously. Because these processes sometimes make requests faster than they can be serviced by the disk, waiting lines or queues build up to hold disk requests. Some early computing systems simply serviced these requests on a **first-come-first-served**

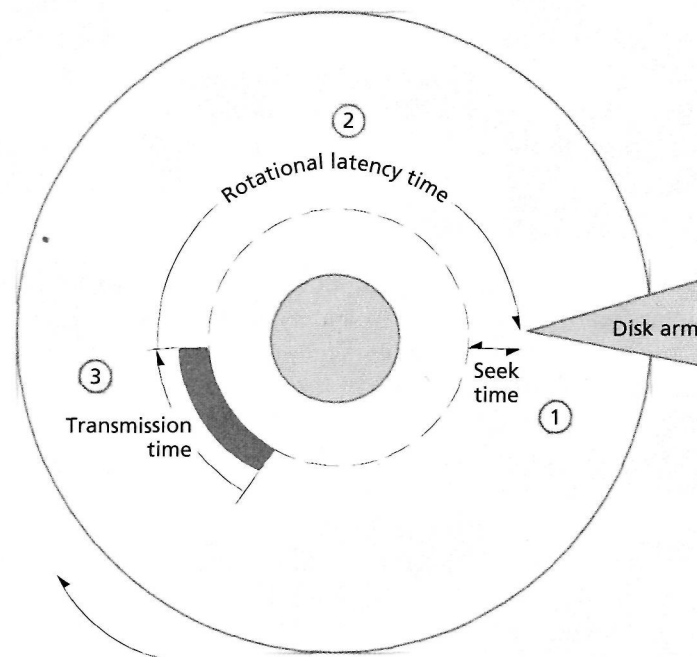


Figure 12.4 | Components of a disk access.

(FCFS) basis, in which the earliest arriving request is serviced first.¹⁹ FCFS is a fair method of allocating service, but when the **request rate** (i.e., the **load**) becomes heavy, can be long waiting times.

FCFS exhibits a **random seek pattern** in which successive requests can cause time-consuming seeks from the innermost to the outermost cylinders. To reduce the time spent seeking records, it seems reasonable to order the request queue in some manner other than FCFS.²⁰ This process, called **disk scheduling**, can significantly improve throughput.²¹

Disk scheduling involves a careful examination of pending requests to determine the most efficient way to service them. A disk scheduler examines the positional relationships among waiting requests, then reorders the queue so that the requests will be serviced with minimum mechanical motion.

Because FCFS does not reorder requests, it is considered by many to be the simplest disk scheduling algorithm. The two most common types of scheduling are **seek optimization** and **rotational optimization**. Because seek times tend to be greater than latency times, most scheduling algorithms concentrate on minimizing total seek time for a set of requests. As the gap between rotational latency and seek times narrows, minimizing the rotational latency of a set of requests can also improve overall system performance, especially under heavy loads.

Self Review

1. Indicate which type of disk scheduling algorithm is most appropriate in the following scenarios: seek time is significantly greater than latency time, seek time and latency time are nearly equal, seek time is significantly shorter than latency time.
2. Which characteristics of disk geometry are most important to seek optimization and rotational optimization?

Ans: 1) When seek time is significantly greater than latency time, access times are most affected by seek operations, so the system should implement seek optimization. When seek times and latency times are nearly equal, the system can benefit from combining both seek and rotational optimization techniques. If seek time is much shorter than latency time, then access times are most affected by latency time, so the system should concentrate on rotational optimization. Because today's processors are so fast, both forms of optimization should be employed to improve overall performance. 2) Cylinder locations are most important when optimizing seek times, and sector locations are most important when optimizing latency times.

12.5 Disk Scheduling Strategies

A system's disk scheduling strategy depends on the system objectives, but most strategies are evaluated by the following criteria

- *throughput*—the number of requests serviced per unit time
- *mean response time*—the average time spent waiting for a request to be serviced
- *variance of response times*—a measure of the predictability of response times. Each disk request should be serviced within an acceptable time period (i.e., the strategy should prevent indefinite postponement).

Clearly, a scheduling policy should attempt to maximize throughput and minimize the mean response time. Many scheduling policies attempt to accomplish these goals by minimizing the time spent performing lengthy seeks. When throughput and mean response time are optimized, average system performance improves, but individual requests may be delayed.

Variance measures how individual requests are serviced relative to average system performance. The smaller the variance, the more likely it is that most disk requests are serviced after waiting for a similar amount of time. Therefore, variance can be seen as a measure of fairness and of predictability. We desire a scheduling policy that minimizes variance (or at least keeps it at reasonable levels) to avoid erratic service times. In a business-critical system, such as a Web server, a high variance of response times could result in loss of sales if, for example, users' requests to purchase products were indefinitely postponed or suffer lengthy waits. In mission-critical systems, the result of such a delay could be catastrophic.

The following sections describe several common scheduling policies. We use the set of disk requests in Fig. 12.5 to demonstrate the result of each policy on an arbitrary series of requests. The arbitrary series of requests is intended to demonstrate how each policy orders disk requests, it does not necessarily indicate the relative performance of each policy in a real system. In the examples that follow, we

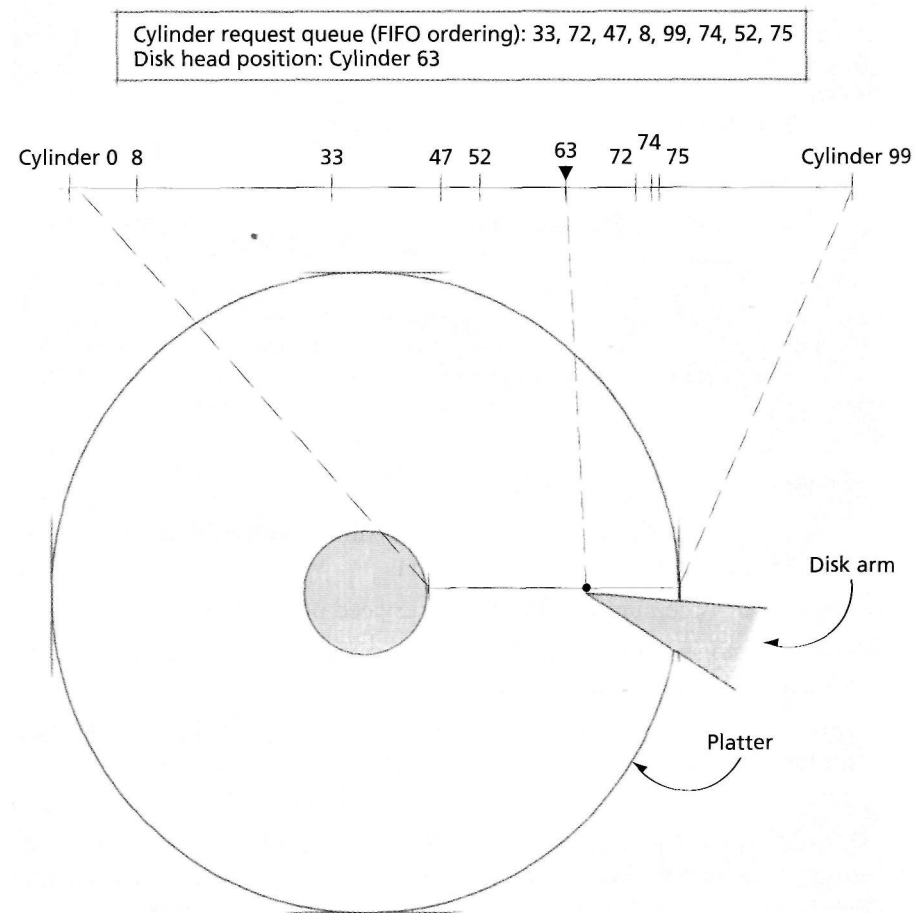


Figure 12.5 | Disk request pattern.

assume that the disk contains 100 cylinders, numbered 0-99, and that the read/write head is initially located at cylinder 63, unless stated otherwise. For simplicity, we also assume that the disk scheduler has already determined the cylinder number corresponding to each request.

Self Review

1. Why is minimizing the variance of response times an important goal in disk scheduling?
2. We mentioned that FCFS is a fair scheduling algorithm. Which of the criteria mentioned in this section most directly relates to fairness?

Ans: 1) Otherwise the system could experience erratic response times. In a business-critical system, this could cause a company to lose customers; in a mission-critical system, it could put people's lives at risk. 2) FCFS is fair in the sense that arriving requests cannot get in line ahead of waiting requests. This helps reduce the variance of response times.

12.5.1 First-Come-First-Served (FCFS) Disk Scheduling

FCFS scheduling uses a FIFO queue so that requests are serviced in the order in which they arrive.^{22,23-24} FCFS is fair in the sense that a request's position in the queue is unaffected by arriving requests. This ensures that no request can be indefinitely postponed, but it also means that FCFS might perform a lengthy seek operation to service the next waiting request, even if another request in the queue is closer and can be serviced faster. Although this technique incurs low execution-time overhead, it can result in low throughput due to lengthy seeks.

When requests are uniformly distributed over the disk surfaces, FCFS scheduling leads to a random seek pattern because it ignores positional relationships among the pending requests (Fig. 12.6). This is acceptable when the load on a disk is light. However, as the load grows, FCFS tends to saturate (i.e., overwhelm) the device, and response times become large. The random seek pattern of FCFS results in low variance (because arriving requests cannot get ahead of waiting requests), but this is of little solace to the request sitting at the back of the disk queue while the disk arm rambles around in a torrid "disk dance."

Self Review

1. One measure of fairness is a low variance in response times. In this sense, is FCFS disk scheduling fair?
2. Can indefinite postponement occur with FCFS disk scheduling? Explain.

Ans: 1) Yes, in this sense it is fair, but because no attempt is made to minimize response times, average response times tend to be longer than necessary. 2) No. Indefinite postponement cannot occur because arriving requests can never be placed ahead of requests in the queue.

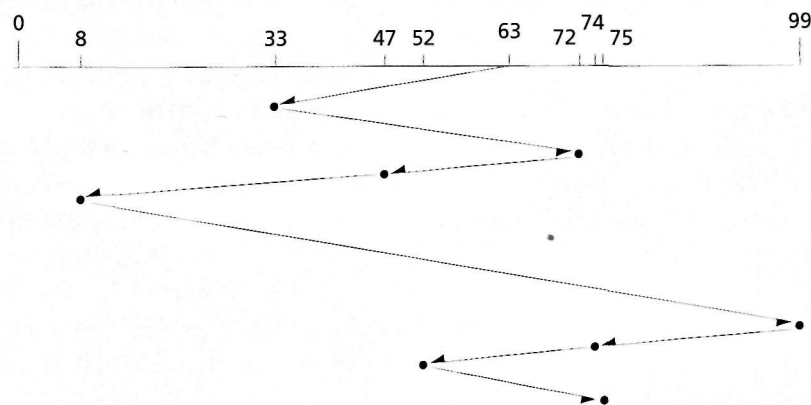


Figure 12.6 | Seek pattern under the FCFS strategy.

12.5.2 Shortest-Seek-Time-First (SSTF) Disk Scheduling

Shortest-seek-time-first (SSTF) scheduling next services the request that is closest to the read-write head's current cylinder (and thus incurs the shortest seek time), even if that is not the first one in the queue.^{25, 26, 27, 28} SSTF does not ensure fairness and can lead to indefinite postponement because its seek pattern tends to be highly localized, which can lead to poor response times for requests to the innermost and outermost tracks (Fig. 12.7).

By reducing average seek times, SSTF achieves higher throughput rates than FCFS, and mean response times tend to be lower for moderate loads. One significant drawback is that it leads to higher variances of response times because of the discrimination against the outermost and innermost tracks; in the extreme, starvation (see Chapter 7) of requests far from the read/write heads could occur if new arrivals tend to be clustered near the middle cylinders. The high variance is acceptable in batch-processing systems, where throughput and mean response times are more important goals. However, SSTF is inappropriate for interactive systems, where the system must ensure that each user experiences prompt, predictable response times (see the Anecdote, Every Problem Has a Solution and Every Solution Has a Problem).



Anecdote

Every Problem Has a Solution and Every Solution Has a Problem

This true story happened to a neighbor who is in no way involved with the computer industry. Her husband left for work shortly before she did and he forgot to close the garage door that day. When she was leaving her home to go to work, she walked into her garage and found a skunk had pushed over a garbage pail and was sitting next to the door to her car merrily

chomping on the garbage. She called the town's animal control department, but they had a backlog and said they would not be able to come out for a few hours. They told her, though, that skunks love bacon, so she should fry up a batch of bacon and throw it out on her front lawn. The skunk would smell the bacon and go out on the front lawn to eat it. She could then hop in her car, pull out

of the garage, close the garage door with her remote control and head off to work. She followed that advice. But much to her chagrin, when she threw the bacon onto the front lawn, another skunk came out of the woods to eat the bacon, while the first skunk continued eating the garbage in her garage. She got to work a few hours late that day.

Lesson to operating systems designers: Always try to anticipate the consequences of your design choices.

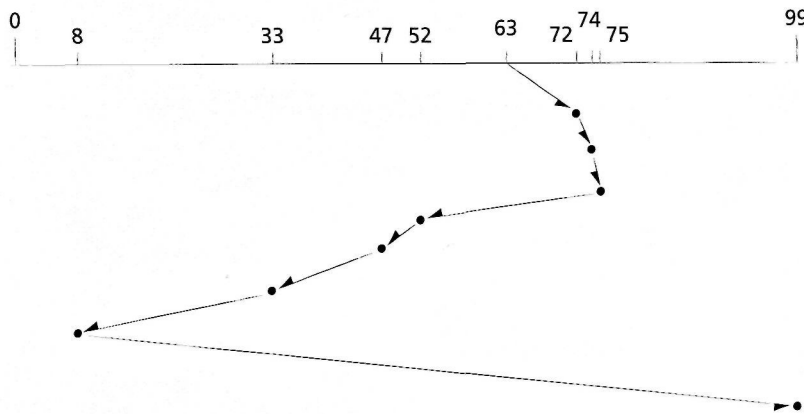


Figure 12.7 | Seek pattern under the SSTF strategy.

Self Review

1. To which scheduling algorithm from Chapter 8 is SSTF most similar? How do they differ?
2. Compare the overhead incurred by SSTF and FCFS.

Ans: 1) SSTF is most similar to the shortest-process-first (SPF) strategy. Unlike SPF, SSTF does not require future knowledge (such as runtime-to-completion)—the request's location is sufficient information to determine which request to service next. 2) Both FCFS and SSTF must maintain a queue of waiting requests in memory. Because FCFS maintains a simple FIFO queue, the execution time required to determine which request to service next is small and is unaffected by the number of requests in the queue. SSTF incurs overhead by either maintaining a sorted list (or lists) of requests or by searching the entire request queue for the appropriate request to service. In this case, the overhead incurred by SSTF is proportional to the number of requests in the queue (whereas the overhead of FCFS is constant—simply the amount of time it takes to insert entries at the back of the queue and remove them from the front).

12.5.3 SCAN Disk Scheduling

Denning developed the **SCAN** disk scheduling strategy to reduce the unfairness and variance of response times exhibited by SSTF.²⁹ SCAN chooses the request that requires the shortest seek distance in a **preferred direction** (Fig. 12.8). Thus, if the preferred direction is currently outward, the SCAN strategy chooses the shortest seek distance in the outward direction. SCAN does not change its preferred direction until it reaches the outermost cylinder or the innermost cylinder. In this sense, it is called the **elevator algorithm**, because an elevator continues in one direction servicing requests before reversing direction.

SCAN behaves much like SSTF in terms of high throughput and good mean response times. However, because SCAN ensures that all requests in a given direction will be serviced before the requests in the opposite direction, it offers a lower variance of response times than SSTF. SCAN, like SSTF, is a cylinder-oriented strategy.

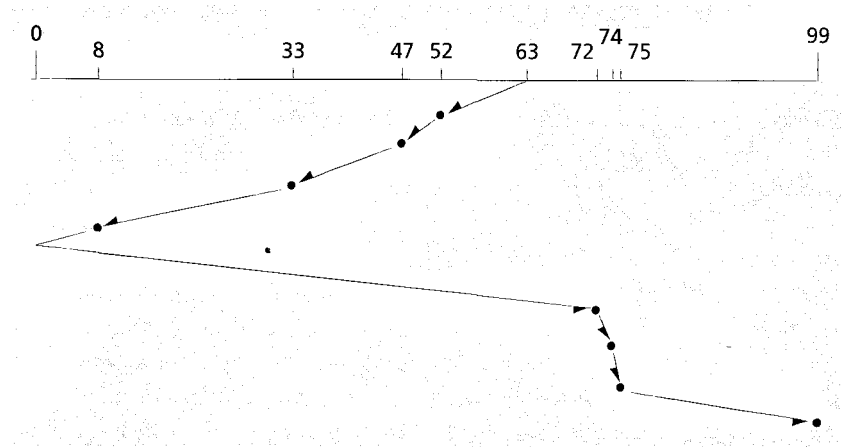


Figure 12.8 | Seek pattern under the SCAN strategy.

Because the read-write heads oscillate between opposite ends of each platter in SCAN, the outer tracks are visited less often than the midrange tracks, but generally more often than they would be using SSTF. Because arriving requests can be serviced before waiting requests, both SSTF and SCAN can suffer indefinite postponement.

Self Review

1. One limitation of the SCAN algorithm is that it might perform unnecessary seek operations. Indicate where this occurs in Fig. 12.8.
2. Can requests be indefinitely postponed under SCAN?

Ans: **1)** The unnecessary seek operation occurs after servicing the request to cylinder 8. Because there are no further requests in the preferred direction, it would be more efficient to change direction after servicing that request. We examine a modification to the SCAN strategy that addresses this limitation in Section 12.5.6, LOOK and C-LOOK Disk Scheduling. **2)** One could imagine a scenario in which processes issue continuous requests to the same cylinder, so that the read/write head becomes "stuck" on that cylinder.

12.5.4 C-SCAN Disk Scheduling

In the **Circular SCAN (C-SCAN)** modification to the SCAN disk scheduling strategy, the arm moves from the outer cylinder to the inner cylinder, servicing requests on a shortest-seek basis (Fig. 12.9). When the arm has completed its inward sweep, it jumps (without servicing requests) to the outermost cylinder, then resumes its inward sweep, processing requests. C-SCAN maintains high levels of throughput while further limiting variance of response times by avoiding discrimination against the innermost and outermost cylinders.^{30,31} As in SCAN, requests in C-SCAN can be indefinitely postponed if requests to the same cylinder continuously arrive (although this is less likely than with SCAN or SSTF). In the sections that follow, we discuss modifications to SCAN that address this problem.

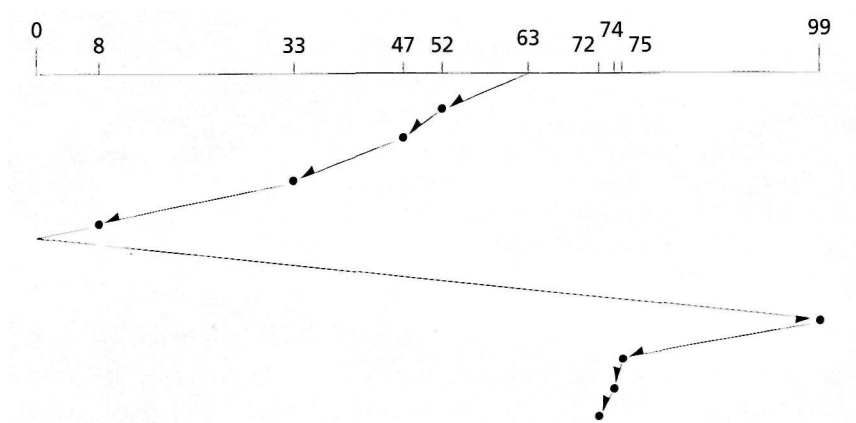


Figure 12.9 | Seek pattern under the C-SCAN strategy.

Simulation results in the literature indicate that the best disk scheduling policy might operate in two stages.³² Under a light load, the SCAN policy is best. Under medium to heavy loads, C-SCAN and other adaptations to the SCAN policy yield better results. C-SCAN with rotational optimization handles heavy loading conditions effectively.³³

SelfReview

1. Which criteria of disk scheduling strategies does C-SCAN improve upon?
2. (T/F) C-SCAN incurs essentially the same execution-time overhead as SCAN.

Ans: 1) C-SCAN reduces variance of response times compared to SCAN because it is less biased against the outermost and innermost cylinders. 2) False. C-SCAN incurs overhead over SCAN when it finishes its inward sweep and skips requests while moving the head to the outermost cylinder.

12.5.5 FSCAN and N-Step SCAN Disk Scheduling

The **FSCAN** and **TV-Step SCAN** modifications to the SCAN strategy eliminate the possibility of indefinitely postponing requests.^{34, 35} FSCAN uses the SCAN strategy to service only those requests waiting when a particular sweep begins (the "F" stands for "freezing" the request queue at a certain time). Requests arriving during a sweep are grouped together and ordered for optimum service during the return sweep (Fig. 12.10).

TV-Step SCAN services the first n requests in the queue using the SCAN strategy. When the sweep is complete, the next n requests are serviced. Arriving requests are placed at the end of the request queue (Fig. 12.11). TV-Step SCAN can be tuned by varying the value for n . When $n=1$, N-Step SCAN degenerates to FCFS. As n approaches infinity, TV-Step SCAN degenerates to SCAN.

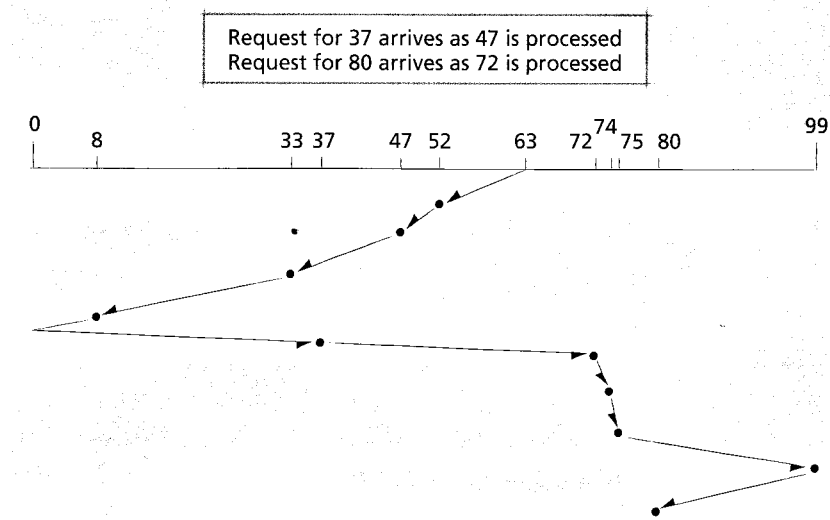


Figure 12.10 | Seek pattern under the FSCAN strategy.

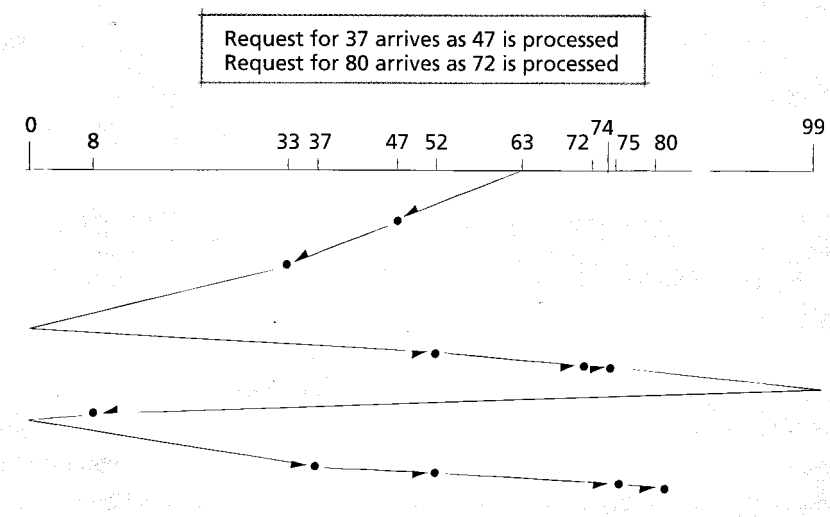


Figure 12.11 | Seek pattern under the N-Step SCAN strategy ($n = 3$).

FSCAN and TV-Step SCAN offer good performance due to high throughput and low mean response times. Because they prevent indefinite postponement, their distinguishing characteristic is a lower variance of response times than with SSTF and SCAN, especially when requests would be indefinitely postponed under the latter two strategies.

Sel Review

1. Explain how FSCAN can lead to lower throughput than SCAN.
2. Compare and contrast FSCAN and JV-Step SCAN.

Ans: 1) If a request to a cylinder in the preferred direction arrives after FSCAN freezes the queue, the disk head will pass by the cylinder without servicing the request until the next pass. 2) Both strategies use SCAN to service a portion of the request queue to prevent indefinite postponement. FSCAN services all the requests in the queue before beginning a sweep in a new preferred direction; TV-Step SCAN services only the next n requests in the queue.

12.5.6 LOOK and C-LOOK Disk Scheduling

The **LOOK** variation of the SCAN strategy "looks" ahead to the end of the current sweep to determine the next request to service. If there are no more requests in the current direction, LOOK changes the preferred direction and begins the next sweep (Fig. 12.12). In this sense, it is appropriate to call this the elevator algorithm, because an elevator continues in one direction until it reaches the last request in that direction, then reverses direction. This strategy eliminates unnecessary seek operations exhibited by other variations of the SCAN strategy (compare the left side of Fig. 12.12 to the corresponding location in Fig. 12.8).

Circular LOOK (C-LOOK) variation of the LOOK strategy uses the same technique as C-SCAN to reduce the bias against requests located at the extreme ends of the platters. When there are no more requests on a current inward sweep, the read/write head moves to the request closest to the outer cylinder (without servicing requests in between) and begins the next sweep. The C-LOOK policy is characterized by a potentially lower variance of response times compared to LOOK and high throughput (although generally lower than LOOK).³⁶ Figure 12.13 summarizes each of the disk scheduling strategies we have discussed.

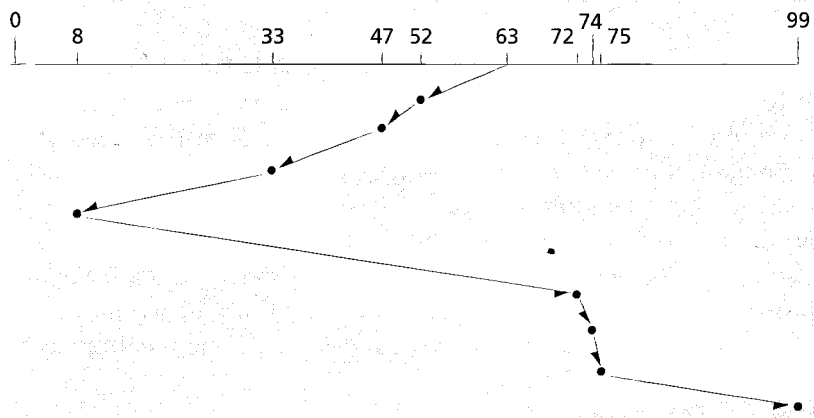


Figure 12.12 | Seek pattern under the LOOK strategy.

<i>Strategy</i>	<i>Description</i>
FCFS	Serves requests in the order in which they arrive.
SSTF	Serves the request that results in the shortest seek distance first.
SCAN	Head sweeps back and forth across the disk, servicing requests according to SSTF in a preferred direction.
C-SCAN	Head sweeps inward across the disk, servicing requests according to SSTF in the preferred (inward) direction. Upon reaching the innermost track, the head jumps to the outermost track and resumes servicing requests on the next inward pass.
FSCAN	Requests are serviced the same as SCAN, except newly arriving requests are postponed until the next sweep. Avoids indefinite postponement.
<i>N</i> -Step SCAN	Serves requests as in FSCAN, but services only <i>n</i> requests per sweep. Avoids indefinite postponement.
LOOK	Same as SCAN except the head changes direction upon reaching the last request in the preferred direction.
C-LOOK	Same as C-SCAN except the head stops after servicing the last request in the preferred direction, then services the request to the cylinder nearest the opposite side of the disk.

Figure 12.13 | *Seek optimization strategies summary.*

SelfReview

1. Why does C-LOOK typically exhibit throughput lower than that of LOOK?
2. Under C-LOOK, in what order would the requests presented in Fig. 12.5 be serviced?

Ans: 1) The time spent seeking from one end of the platter to the furthest request from the position of the read/write head at the end of the sweep increases the mean response time, which decreases throughput. 2) 52, 47, 33, 8, 99, 75, 74, 72

12.6 Rotational Optimization

Because the dominant component of access time in early hard drives was seek time, research focused on seek optimization. However, today's hard disks exhibit seek times and average latencies that are of the same order of magnitude, meaning that rotational optimization can often improve performance.³⁷ Processes that access data sequentially tend to access entire tracks of data and thus do not benefit much from rotational optimization. However, when there are numerous requests to small pieces of data randomly distributed throughout the disk's cylinders, rotational optimization can improve performance significantly. In this section, we discuss how to combine seek and rotational optimization strategies to achieve maximum performance.

12.6.1 SLTF Scheduling

Once the disk arm arrives at a particular cylinder, there might be many requests pending on the various tracks of that cylinder. The **shortest-latency-time-first**

(**SLTF**) strategy examines all of these requests and services the one with the shortest rotational delay first (Fig. 12.14). This strategy has been shown to be close to the theoretical optimum and is relatively easy to implement.³⁸ Rotational optimization is sometimes referred to as **sector queuing**; requests are queued by sector position around the disk, and the nearest sectors are serviced first.

Self Review

1. Why is sector queuing easy to implement?
2. Is rotational optimization appropriate for today's hard disks? Why?

Ans: 1) Sector queuing determines the best access pattern for sectors on a track. Assuming that sectors are at fixed locations and that the disk can spin only in one direction, sector queuing simply is a sorting problem. 2) Yes, rotational optimization is appropriate. Today's hard disks exhibit seek times and average latencies that are of the same order of magnitude.

12.6.2 SPTF and SATF Scheduling

The **shortest-positioning-time-first (SPTF)** strategy next services the request that requires the shortest **positioning time**, which is the sum of the seek time and rotational latency time. Like SSTF, SPTF results in high throughput and a low mean

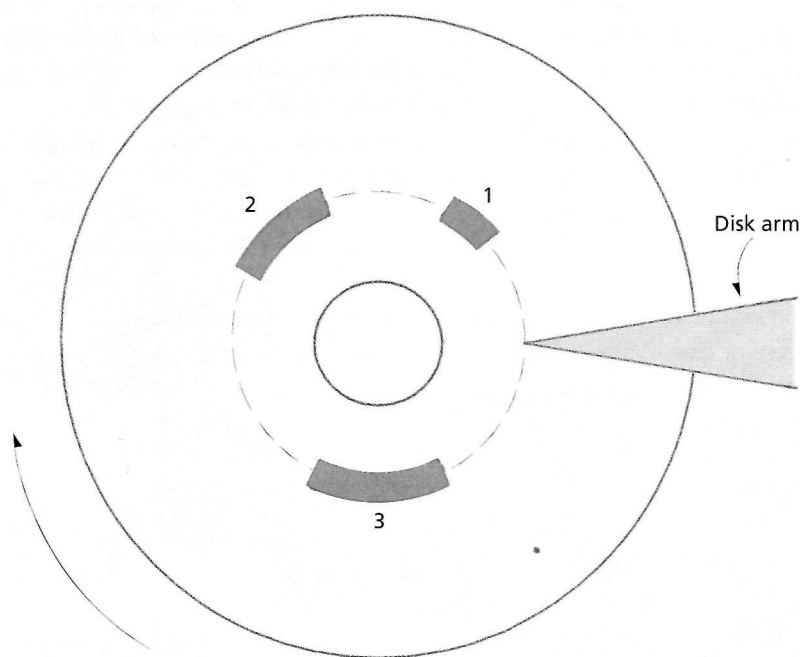


Figure 12.14 | SLTF scheduling. The requests will be serviced in the indicated order regardless of the order in which they arrived.

response time, and it can also indefinitely postpone requests to the innermost and outermost cylinders.³⁹

A variation of SPTF is the **shortest-access-time-first (SATF)** strategy, which next services the request that requires the shortest access time (i.e., positioning time plus transmission time). SATF exhibits higher throughput than SPTF, but large requests can be indefinitely postponed by a series of smaller requests, and requests to the innermost or outermost cylinders can be indefinitely postponed by requests to midrange cylinders. Both SPTF and SATF can improve performance by implementing the "look-ahead" mechanisms described in Section 12.5.6.⁴⁰

Figure 12.15 demonstrates the difference between SPTF and SATF. In Fig. 12.15(a), the disk receives two requests for data records of the same size, A and B, that are located on adjacent cylinders. Data record A is located in the same cylinder as the read/write head, but approximately one-half rotation away from the read/write head. Data record B is near the read/write head but is located on an adjacent cylinder. In this case, the transmission time is identical for A and B because A and B are the same size, so SATF reduces to SPTF. Thus, for this particular disk, both SATF and SPTF would service the request for data record B first, because the disk takes less time to perform a single-cylinder seek operation than to rotate the platter 180 degrees. If the seek time is adequately small, the disk can service the request for B, then reposition the head to service the request for A within one disk rotation. On the contrary, SSTF would service A first, requiring more than one revolution of the platter to service both requests. Thus, SATF and SPTF can increase throughput compared to SSTF

Now consider that data record B consumes an entire track whereas A is stored in one sector, as shown in Fig. 12.15(b). Assume that the location of the first byte of

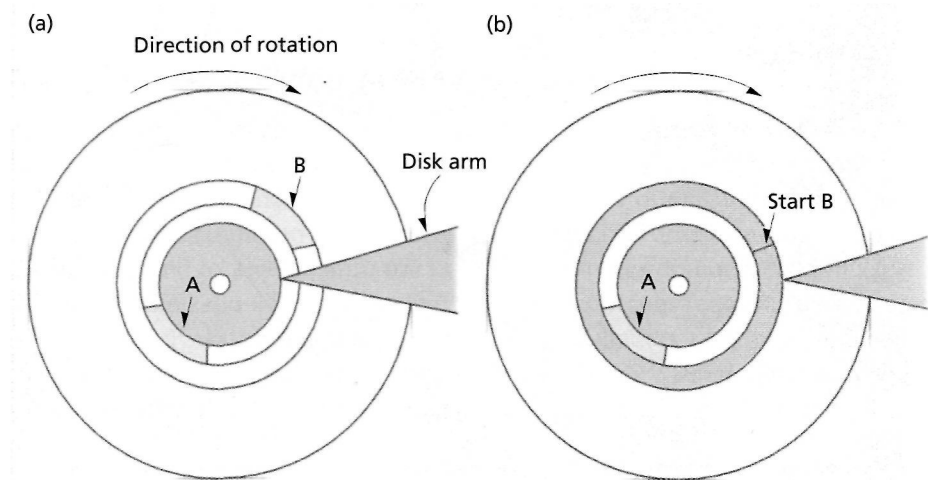


Figure 12.15 | SPTF (a) and SATF (b) disk scheduling examples.

each record is the same as in the previous example. In this case, SPTF would service requests in exactly the same order as in the previous example, because the positioning times are the same. However, SATF would service request A first, because request A requires one half of a rotation plus nominal transmission time, whereas request B requires a nominal rotational latency time plus a transmission time equal to the time required for the platter to complete one revolution.

SPTF and SATF require knowledge of disk characteristics including latency time, track-to-track seek times and relative locations of sectors. Unfortunately, many of today's hard drives expose misleading geometry. For example, many disks hide error-checking and correcting data from the operating system (so that it cannot be unintentionally or maliciously modified), so consecutively numbered sectors may not precisely correspond to contiguous physical locations on disk. On some systems, when bad sectors are detected, alternate ones are assigned. For example, if sector 15 becomes unusable, the disk might send its requests to a reserved sector other than 14 or 16. These alternate sectors can be widely dispersed over the disk surfaces, causing seeks where none might actually be expected.

These features, such as using alternate sectors, which improve data integrity, tend to counteract efforts to improve disk performance using scheduling strategies, because the disk provides the operating system with incomplete or misleading positional information.⁴¹ Although some architectures and hard disks provide commands to retrieve the real geometry of the disk, this feature is not supported by all disks. Other hard disk architectural features complicate disk scheduling strategies; for more information, see the links provided in the Web Resources section at the end of the chapter.

A straightforward performance enhancement technique is to decrease rotational latency by increasing the rotational speed of the disk. However, designers have encountered significant problems when increasing the rotations per minute (RPM) of a hard disk. Faster-rotating disks consume greater amounts of energy (a key concern on laptop computers with limited battery power), radiate more heat, make more noise and require more costly mechanical and electrical mechanisms to control them.⁴² Given these and other problems, RPM speed has increased only modestly (only a few percent a year over the past decade) to approximately 5,400 to 7,200 RPM for PCs and 10,000-15,000 RPM for servers and high-end machines.⁴³

Self Review

1. What is the difference between the SPTF and SATF algorithms? Why would a designer choose one over the other?
2. What factors prevent operating systems from obtaining accurate knowledge of hard disk geometries?

Ans: 1) SPTF services the request that it can begin accessing within the shortest period of time. SATF services the request that it can complete accessing within the shortest time. Although SATF has higher throughput, it favors requests to small amounts of data over requests to large amounts of data. Thus, SATF is more appropriate for systems that issue